

第2章 应用层

主要内容

2.1 网络应用原理

2.2 Web和HTTP

2.3 电子邮件

2.4 DNS

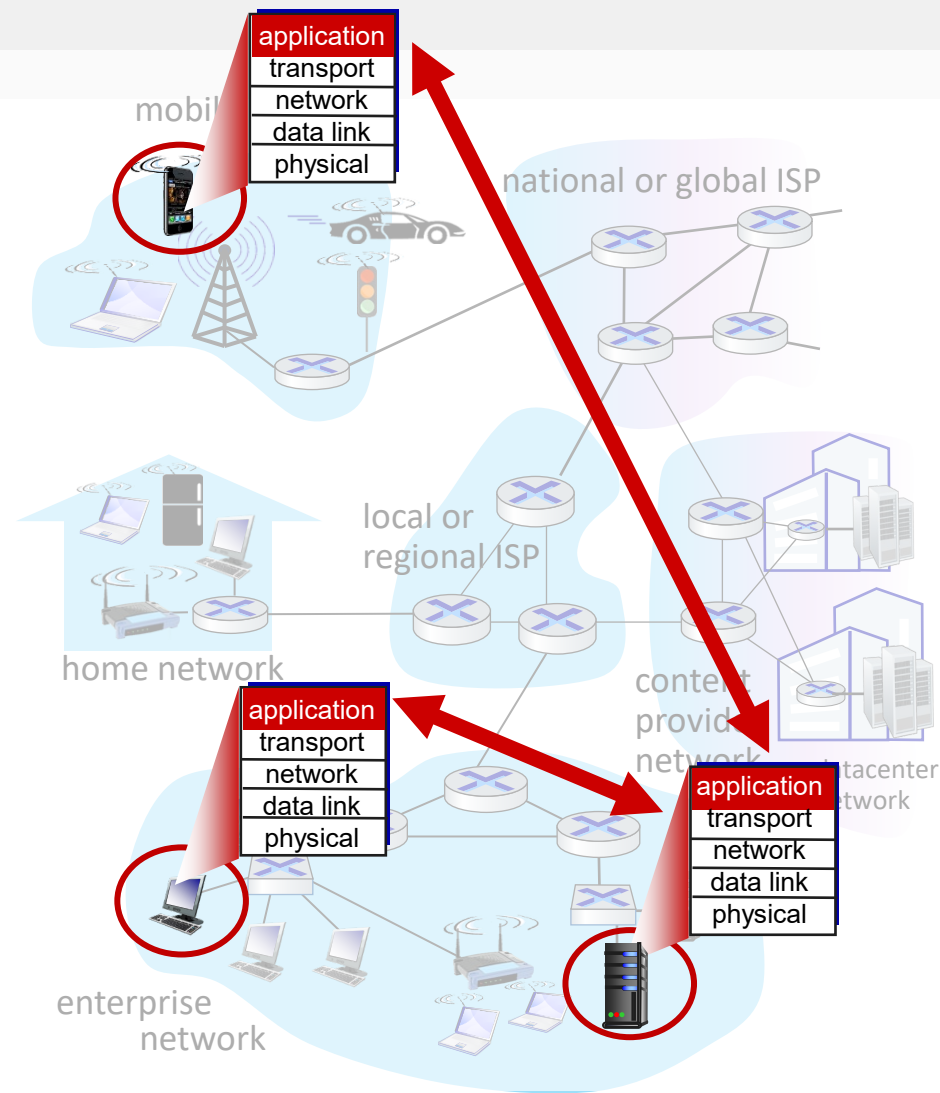
2.5 P2P文件分发

2.6 视频流和内容分发网

2.7 套接字编程

网络应用

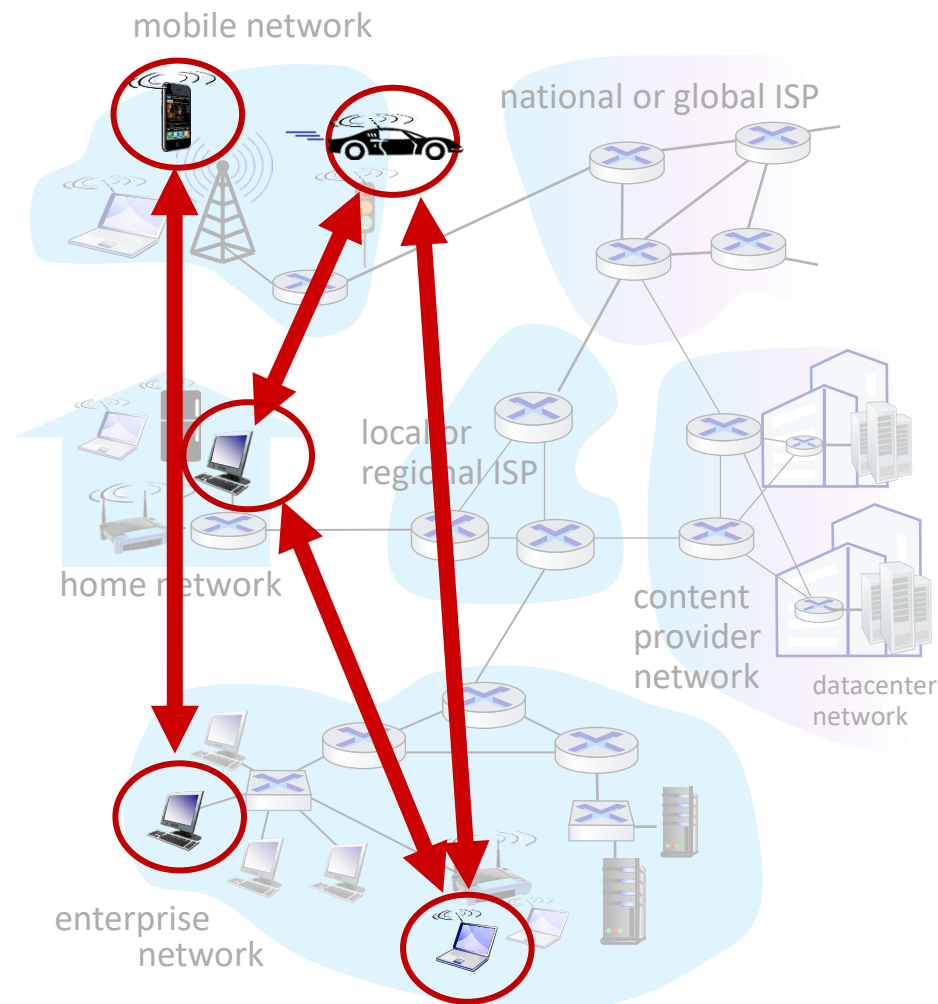
- 编写网络应用：
 - 只需要在各种端系统中实现
 - 不需要（绝大部分情况下也不可能）在网络设备上编程
- 应用体系结构：如何在端系统上组织应用程序
 - **客户-服务器体系结构C/S**(Client-Server Architecture)
 - 服务方：经常部署在数据中心
 - 拥有**固定的IP地址**，服务方程序一直运行
 - **被动等待**来自客户的请求
 - 可“同时”处理多个请求
 - 大量请求时可通过负载均衡技术由多个服务器其中一个提供服务
 - 客户方：
 - 客户方**主动请求**服务器的服务
 - 不需要一直在线，不需要固定IP地址
 - 客户方之间不直接通信



大部分网络应用都采用C/S模型

网络应用

- 客户-服务器体系结构C/S(Client-Server Architecture)
- **P2P体系结构**(Peer to Peer Architecture)
 - 不需要有一个一直运行的服务器
 - peer不需要一直连接，可间断连接，可以改变IP地址
 - peer之间可以直接进行平等、对等的通信
 - peer**同时充当Client和Server的角色**
 - peer可以请求其他peer提供的服务
 - peer也可以为其他peer提供服务
 - **自扩展性(self scalability):**
 - 新的peer带来新的服务需求的同时，也可以为其他peer提供服务
 - BitTorrent: P2P文件共享

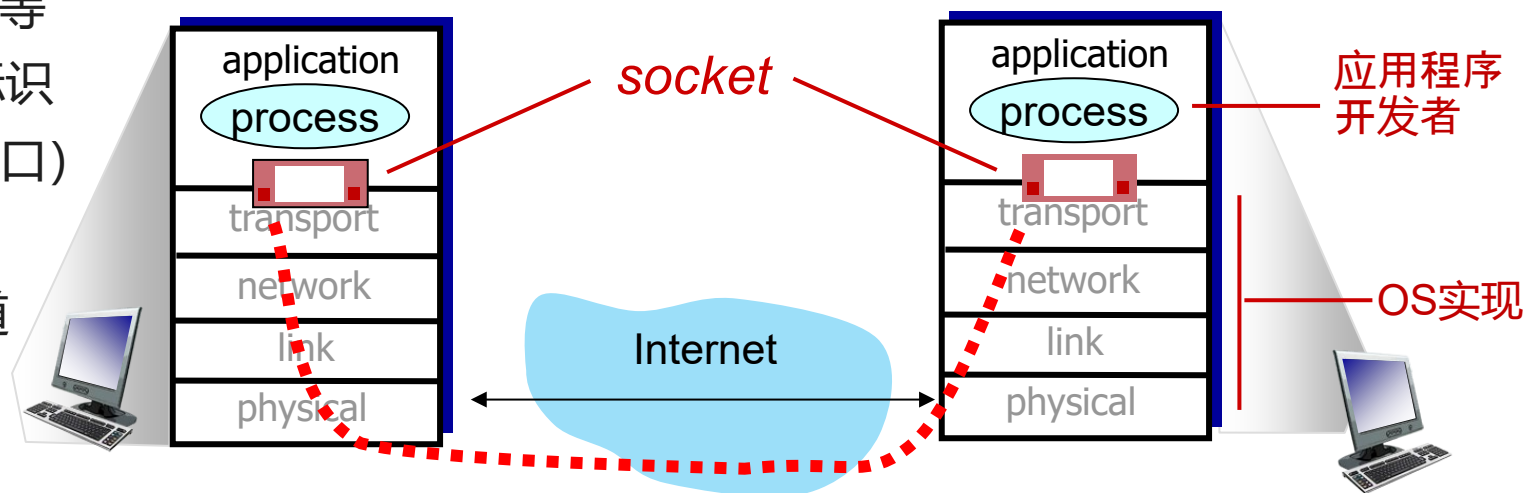


- 客户-服务器体系结构C/S(Client-Server Architecture)
- P2P体系结构(Peer to Peer Architecture)
- **浏览器-服务器体系结构B/S**(Browser-Server Architecture):
 - C/S架构的一种特殊情形: Browser=Client, Web Server = Server
 - 瘦客户-胖服务器: 用户界面由Web浏览器提供, 一部分事务逻辑在前端实现, 主要的事务逻辑在服务器端实现

进程间通信：socket

进程：某个主机上正在运行的程序

- 同一个主机上的两个进程间的进程间通信：管道、消息队列、共享内存+信号量等IPC机制
- 不同主机（也包括同一个主机）的两个进程间的通信：通过网络交换消息
 - 发起通信的一方称为客户方进程，等待联系的一方称为服务方进程
 - **套接字(socket)**：操作系统提供的网络编程的API接口
 - 通过socket接口使用**底层协议**（一般为传输层协议）所提供的服务：通过网络发送和接收消息
 - 底层可以是提供可靠数据传输服务的TCP
 - 底层可以是提供不可靠数据传输服务的UDP
 - 底层甚至可以是网络层的IP协议等
 - 每个Socket通过IP地址+端口号唯一标识
 - (32bit)**IP地址**：哪个主机(网络接口)
 - (16bit)**端口号**：哪个进程
 - 底层提供的服务可看成一条逻辑通道
 - 一对socket对应着该通道两端
 - 一个进程可以同时使用多个Socket



网络应用所要求的服务

- 数据传输的可靠性
 - 有些应用（文件传输、电子邮件、Web浏览等）要求可靠的数据传输
 - 有些容忍丢失的应用(loss-tolerant application)能够承受一定的数据丢失，比如多媒体应用
- 吞吐率
 - 带宽敏感应用(bandwidth-sensitive application) 有最低的吞吐率要求。比如多媒体应用
 - 弹性应用(elastic application)利用可用的带宽，并没有特定的吞吐率要求
- 时间(timing)
 - 有些应用(交互式音视频，在线游戏等交互式实时应用) 要求低延迟
 - 非实时应用对延迟没有严格的约束
- 安全
 - 加密，消息完整性，端点认证等

Internet的传输层协议提供的服务

TCP协议提供的服务

- 面向连接：数据传输之前必须首先建立连接
- 可靠数据传输
- 流量控制：接收者控制发送者的发送速度
- 拥塞控制：网络拥塞时控制发送者的发送速度
- 不提供的服务：**最低延迟保证，最小吞吐率保证，安全**

UDP协议提供的服务

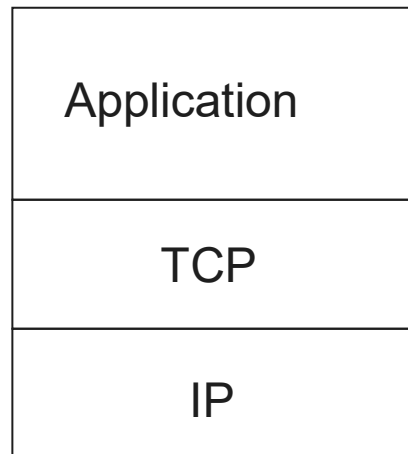
- 不可靠的数据传输
- 无连接方式：不需要实现建立连接
- 不提供的服务：可靠传输，流量控制，拥塞控制，**延迟保证，吞吐率保证，安全**

如果需要安全、延迟保证和吞吐率保证呢？

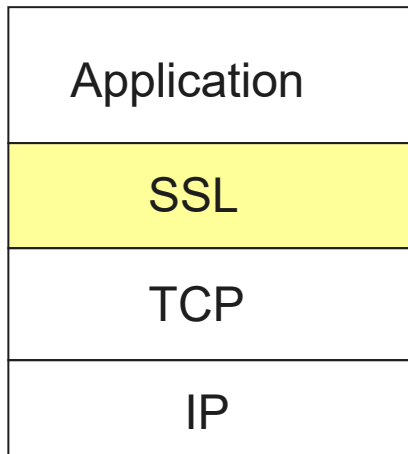
- **安全**：TLS over TCP，DTLS over UDP
- 延迟和吞吐率：Internet只是没有提供保证，但实践中大多数情况能够满足那些时间和带宽敏感应用的需求

保护TCP连接

- 传统的应用层协议采用基于7bit ASCII字符集的请求响应协议，消息（包括口令）明文传递
- SSL和TLS：提供用户认证、消息完整性和消息加密支持
 - Secure Socket Layer：Netscape提出，目前的版本SSL 3.0，已不建议使用(RFC 7568)
 - Transport Layer Security: RFC 2246定义TLS1.0，在SSL 3.0的基础上进一步改进。RFC 5246定义了TLS1.2
 - 2018年9月RFC 8446定义了TLS1.3
 - 人们提起SSL，现在一般指TLS
- 隐式的Implicit Application over TLS：**仍然采用原有的应用层协议**，
 - 首先连接到**Secure Application**端口，采用TLS保护连接
 - 在采用TLS保护的安全连接上采用Application协议交互
 - 整个Application协议都被保护
- 显式的Explicit Application over TLS：**对原有的应用层协议扩展**
 - 首先连接到**原有的Application**端口
 - 对原有应用层协议进行扩展，发送请求(比如STARTTLS或AUTH TLS)开启TLS支持
- 建议优先采用Implicit Application over TLS



应用



应用 over TLS

应用	可靠性	带宽	时间敏感	应用层协议	传输层协议
远程终端访问	不能丢失	弹性	是, 几百毫秒	Telnet	TCP
文件传输	不能丢失	弹性	不	FTP	TCP
电子邮件	不能丢失	弹性	不	SMTP	TCP
Web文档	不能丢失	弹性(few kbps)	不	HTTP	TCP
Internet音频(电话)	容忍丢失	few kbps-1Mbps	是, 几百毫秒	SIP, RTP, 私有协议	UDP或TCP
Internet视频		10kbps-5Mbps			
流式存储音视频	容忍丢失	同上	是, 几秒	HTTP, DASH	TCP
交互式游戏	容忍丢失	few kbps-10kbps	是, 几百毫秒		UDP或TCP
智能手机信息	不能丢失	弹性	Yes and no		TCP

应用层协议

- 应用层协议：具体的应用程序在通信时应该遵循的协议
 - 交换的消息类型：比如请求和响应
 - 消息的语法：哪些字段，每个字段的格式如何
 - 消息的语义：每个字段的含义
 - 消息发送和响应的规则：何时以及如何发送消息，怎么响应收到的消息
- 应用层协议只是网络应用的一部分
- 开放协议：
 - 在每个人都可以访问的RFC文档中定义
 - 不同的实现之间可以互联
- 私有协议：
 - 厂商内部制定的协议，不公开，比如腾讯会议、Zoom等

主要内容

2.1 网络应用原理

2.2 Web和HTTP

2.3 电子邮件

2.4 DNS

2.5 P2P文件分发

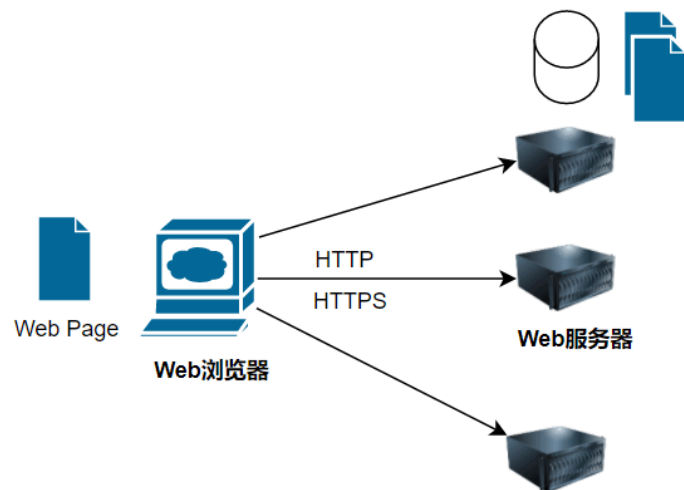
2.6 视频流和内容分发网

2.7 套接字编程

Web体系结构

- Web上有各种资源（也称为Web对象）：HTML文档、图像、视频、音频、脚本、样式表等
- 可以通过Web Page(Web页面、网页) 的形式将多个资源组织在一起
- Web Page一般采用**HTML 语言**(Hypertext Markup Language, 超文本标记语言) 描述
 - 通过**超链接(hyperlink)**将Web页面与其他Web资源（Web页面、图像、音视频、文件等）关联起来，因而称为超文本(Hypertext)
 - HTML语言通过Tag(标签) 标记要显示的网页的各个部分，包括超链接、文本、列表、表格、图片等信息
 - **层叠样式表CSS**(Cascading Style Sheets)描述HTML文本的样式信息，包括字体、颜色和布局等
- 各个Web Page位于某个**Web服务器**（也称为HTTP服务器、网站website）
- Web Client也称为**浏览器**，浏览器采用HTTP协议（Hyper Text Transfer Protocol, 超文本传输协议）与Web服务器交互，请求位于Web服务器的Web Page
- 浏览器在接收到Web Page之后，解析该Web Page，并且将其呈现给用户
- 早期的网页比较小，只有少数几个对象，现在的网页比较大，平均有140个对象

`<a href="http://www.fudan.edu.cn">Fudan University</a>`



统一资源定位符(URL)

- Web资源的访问有3个问题:

- 要访问的是哪个资源?
- 要访问的资源在哪里?
- 怎么样访问资源?

<https://tools.ietf.org/html/rfc2045>

<http://www.urp.fudan.edu.cn:88/index.jsp>

<https://www.baidu.com/s?wd=uri%20urn>

- Uniform Resource Identifier统一资源标识符 (URI): 标识 (抽象或物理的) 资源

- **Uniform Resource Location**统一资源定位符(URL): 是URI的最常见表示形式

scheme:[//[user[:password]@]host[:port]][/path] [?query][#fragment]

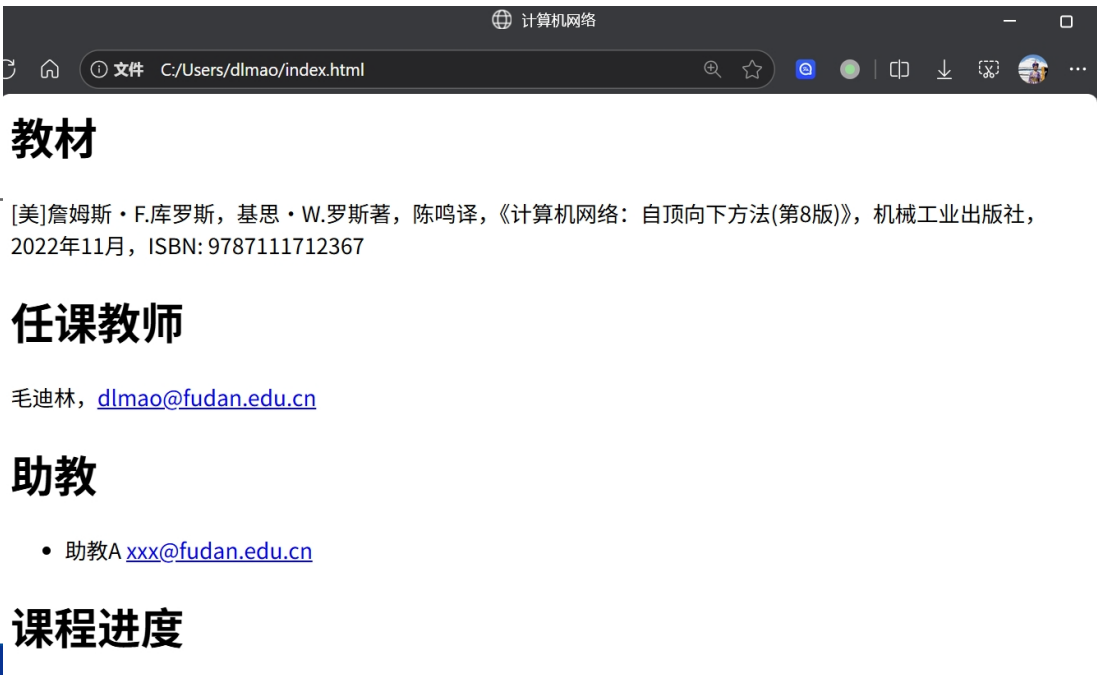
authority

- scheme: 如何访问资源, 比如http(s), ftp, mail, file, rtsp, sip等, 缺省为http协议
- authority: 管理该资源的实体(如所在的主机), 包括可选的端口号、用户名和密码
- path: 给出资源在主机中的位置, 虽然采用类似于文件系统的目录形式, 但并不一定与其对应
- query: 查询字符串, 传递相应的参数来访问该资源。格式为attr=value, 之间以&隔开
- fragment: 描述主资源中的子资源, 由client来解释

```

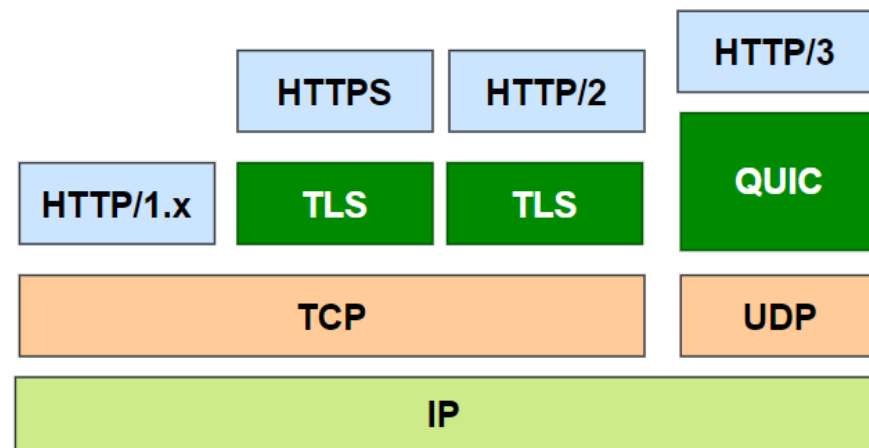
<html>
<head>
  <title>计算机网络</title>
</head>
<body>
  <h1>教材</h1>
  <p> [美]詹姆斯·F.库罗斯, 基思·W.罗斯著, 陈鸣译, 《计算机网络: 自顶向下方法(第8版)》, 机械工业出版社, 2022年11月, ISBN: 9787111712367 </p>
  <h1>任课教师</h1>
  <p>毛迪林, <a href="mailto:dlmao@fudan.edu.cn">dlmao@fudan.edu.cn</a></p>
  <h1>助教</h1>
  <ul>
    <li>助教A <a href="mailto:xxx@fudan.edu.cn">xxx@fudan.edu.cn</a></li>
  </ul>
  <h1>课程进度</h1>
</body>
</html>

```



HTTP (Hyper Text Transfer Protocol, 超文本传输协议)

- 采用TCP, 缺省端口为80
- https表示相应的TCP应该采用TLS保护, 缺省的https端口为443
- **基于文本的请求响应模式**: 客户机向服务器发送HTTP请求, 服务器发送HTTP响应
- 无状态(stateless)的协议
 - HTTP服务器并不维护之前收到的**HTTP请求的状态**
 - 简化了服务器的设计, 容易支持大量并发的HTTP请求
 - HTTP协议提供**cookie机制来传递状态**信息
- 1996年RFC 1945定义了HTTP1.0
- 1997年RFC2068定义了HTTP1.1
- 2015年RFC7540定义了HTTP/2, 与HTTP/1.1并不兼容
- 2022年RFC 9114定义了HTTP/3, 位于新的传输层协议QUIC之上



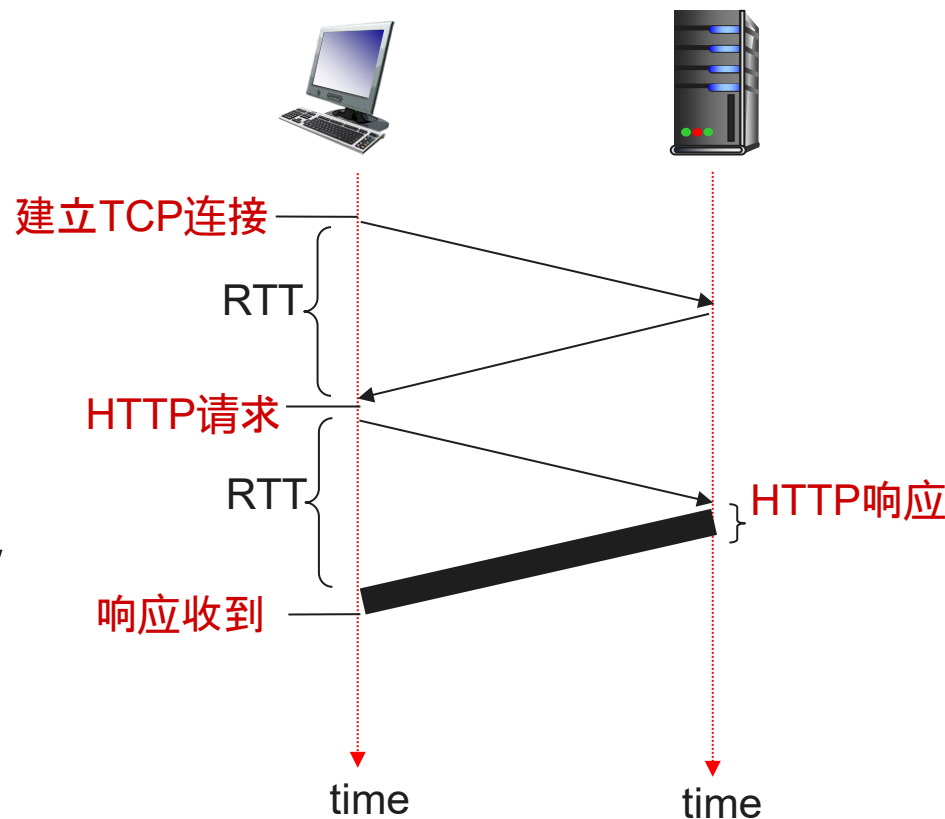
- RFC 9110 HTTP Semantics
- RFC 9111 HTTP Caching
- RFC 9112 HTTP/1.1
- RFC 9113 HTTP/2
- RFC 9114 HTTP/3

HTTP/1.0: 非持续(Non-persistent)连接

HTTP1.0采用非持续(Non-persistent)连接方式:

- 要获得一个Web对象: HTTP请求
 - 新建一条TCP连接
 - 发送请求
 - 服务方发送HTTP响应, 并关闭TCP连接
- TCP连接:
 - 连接建立需要1RTT
 - 新连接的拥塞窗口为1, 即初始不能发送大量数据
- 如果一个HTML页面包含了N个同一服务器上的其他小Web对象, 假设这些对象都可容纳在一个TCP段中 (忽略TCP的拥塞控制)
 - $N+1$ 条连接: HTML页面 + N个对象
 - 响应时间为: $(N+1)*2RTT + \text{传输时间}$

以前一个页面的对象小于10个, 现在的页面平均140个对象

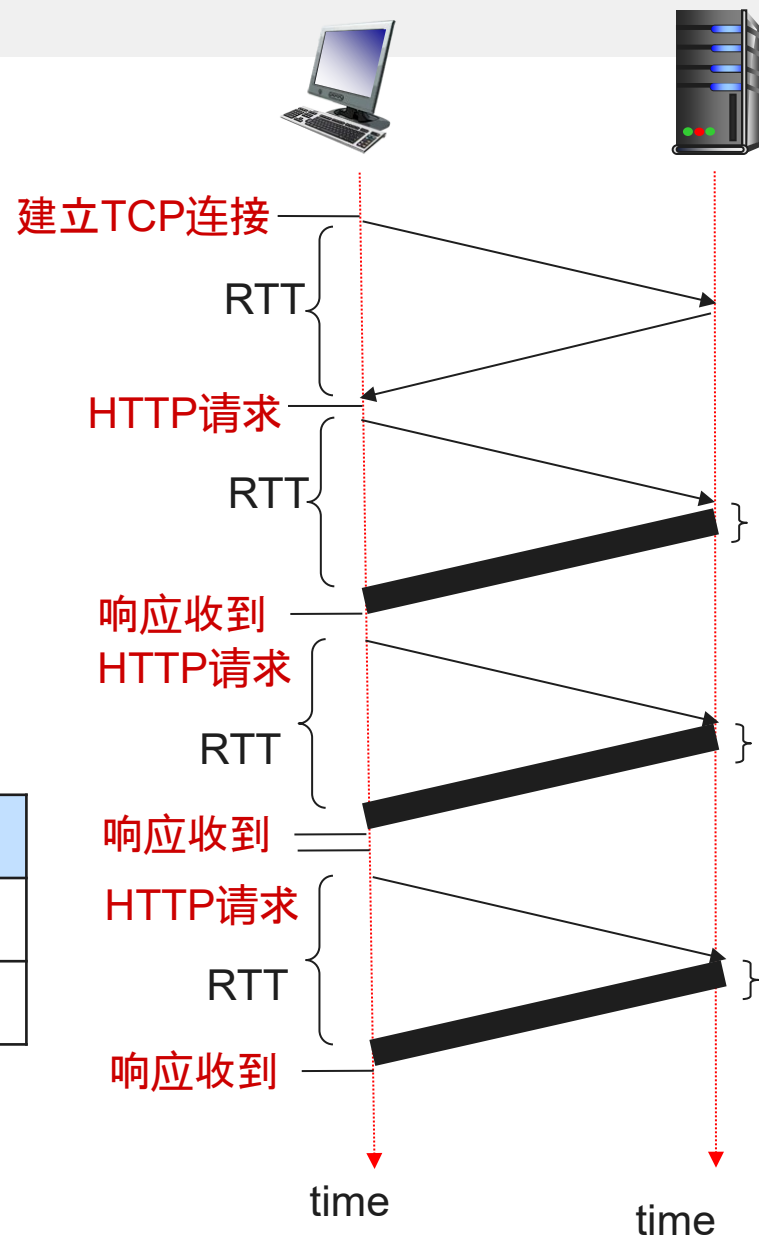


有些浏览器支持**并行连接**方式: 同时建立多条 (一般最多允许6条) TCP连接

HTTP/1.1 持续(persistent)连接

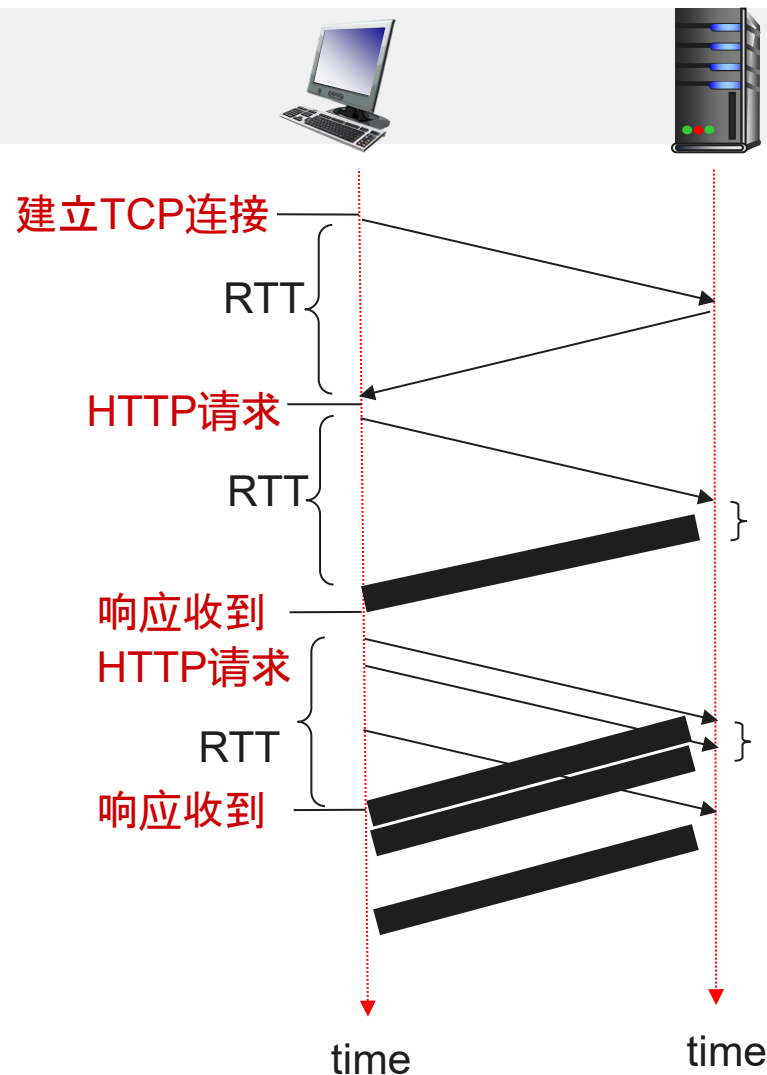
- 在收到HTTP响应时暂时不关闭TCP连接，而是等待一段时间
 - 如果持续连接空闲一段时间 (keep-alive.timeout)，则关闭TCP连接
- 在此期间如果有到该服务器的新请求可以重用该连接
- 如果短时间内有多个到同一个服务器的HTTP请求：
 - 避免连接建立的开销
 - 拥塞窗口也不用每次从初始的1开始
- 如果一个HTML页面包含了N个同一服务器上的其他小的Web对象，响应时间为：

非持续连接	(非流水线形式的) 持续连接
每个请求需要2RTT	第一个请求需要2RTT，后面N请求只要RTT
$(N+1)*2RTT + \text{传输时间}$	$(N+2)RTT + \text{传输时间}$



HTTP/1.1 持续(persistent)连接

- 非流水线方式的持续连接：
 - 发送第一个请求，收到第一个响应
 - 发送第二个请求，收到第二个响应....
- **流水线方式(pipelining, 管道化)**的持续连接
 - 发送第一个请求，收到第一个响应(HTML页面)
 - 发送第2、3、4...个HTTP请求，收到第2、3、4...个HTTP响应
 - 要求**HTTP响应自身能够决定边界**（即结束的位置）
 - 绝大部分浏览器缺省关闭流水线方式
- 队头阻塞(Head of line blocking): 按照请求的顺序发送HTTP响应，如果一个对象较大，阻塞后面的对象



非持续连接	持续连接	流水线方式持续连接
每个请求需要2RTT	第一个请求需要2RTT，后面请求只要RTT	第一个请求2RTT，然后发送所有请求
$(N+1)*2RTT + \text{传输时间}$	$(N+2)RTT + \text{传输时间}$	如果 所有响应 可容纳在 一个 TCP段中 (所有请求 也是如此) $3RTT + \text{传输时间}$

HTTP 请求

请求行

(GET/POST/HEAD **主要方法**
PUT/DELETE/TRACE
OPTIONS/CONNECT)

头部行

空行表示头部部分的结束
\\r\\n\\r\\n

```
GET /2016/index.html HTTP/1.1
Host: www.fudan.edu.cn
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:47.0)
Gecko/20100101 Firefox/47.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: zh-CN,zh;q=0.8,en-US;q=0.5,en;q=0.3
Accept-Encoding: gzip, deflate
Cookie: amlbcookie=01; iPlanetDirectoryPro=xxxxxxxxxxxxxxxx
DNT: 1
Connection: keep-alive
```

基于请求行的URL + Host头部, 实际访问的URL:

http://www.fudan.edu.cn/2016/index.html

来自于Host头部

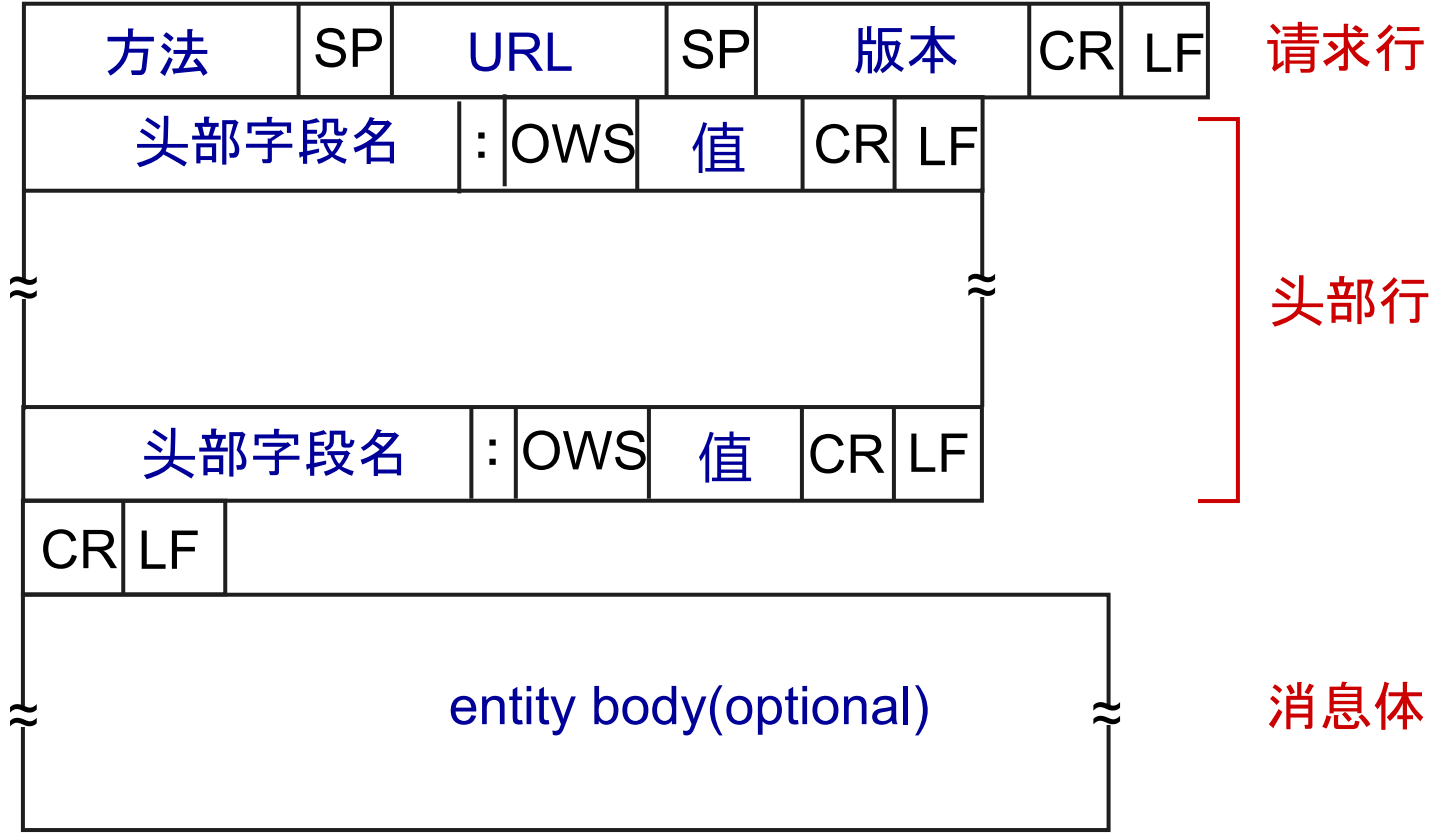
来自于请求行

- 请求行中的URI部分不包含fragment部分, **fragment由Client解释**
- 请求行中的URI一般不包括主机部分, **Host头部**描述是哪个Web站点
- 使用代理(**Proxy**)时, 请求行中的URI中为**完整URI**

```
GET http://mail.qq.com/ HTTP/1.1
Host: mail.qq.com
```

HTTP 请求：格式

- 版本： HTTP/1.0, HTTP/1.1等
- 头部行： 字段名和值之间以冒号 + 可选的空格(OWS)隔开
- 头部部分的结束： 一个空行（即CRLF CRLF）
- 消息体可选： 具体长度根据头部的相应字段决定



方法	描述
GET	获取资源。 必须实现
HEAD	与GET类似，不包括资源的具体内容。 必须实现
POST	常用于表单(form)提交，请求中包含body部分，对资源进行处理
PUT	请求中包含body部分，创建或替换资源
DELETE	删除资源
CONNECT	建立隧道
OPTIONS	询问服务器支持的HTTP请求方法等选项
TRACE	调试，服务器将收到的HTTP请求作为消息体返回

HTTP响应

- Code:状态码, 给出请求处理的结果
- Reason: 可选, 状态码的描述信息

状态行
(Version Code Reason)

状态码	描述
1xx	informational, 还不是最终响应
2xx	success, 请求处理成功
3xx	redirection, 在另一位置或者缓存中获取
4xx	client error, 客户方的错误
5xx	server error, 服务方错误

100 Continue

200 OK

204 No Content

301 Moved Permanently Location头部给出新的位置

304 Not Modified

401 Unauthorized

403 Forbidden

404 Not Found

500 Internal Server Error

505 HTTP Version Not Supported

头部行

HTTP/1.1 200 OK
Server: nginx
Date: Thu, 20 Oct 2016 00:41:27 GMT
Content-Type: text/html
Content-Length: 75643
Last-Modified: Tue, 18 Oct 2016 09:58:11 GMT
ETag: "5805f233-1277b"
Accept-Ranges: bytes
X-Cache: MISS from 2ndDomainSrv
X-Cache-Lookup: MISS from 2ndDomainSrv:80
Via: 1.1 2ndDomainSrv (squid)
Connection: keep-alive

空行表示
头部部分的结束
\r\n\r\n

...<!DOCTYPE HTML>

HTTP 头部

头部 (部分)	含义
Host	服务器可能支持多个虚拟网站, 访问哪个网站上的资源
User-Agent	传递浏览器的相关信息
Accept	客户方可接受的资源类型
Accept-Language	客户方可接受的资源语言
Accept-Encoding	客户方可接受的资源编码(压缩)方式(比如gzip)
Connection	收到HTTP响应后是否关闭, 可取值 keep-alive, close, upgrade等
Cookie	Cookie信息 (来自于之前服务器通过set-cookie发送的信息)
Server	服务器的相关信息
Date	响应发送时的时间
Content-Type	消息体包含的内容的类型。文本类型时可指定charset
Content-Encoding	内容本身采用了指定的编码 (压缩) 方式进行编码
Content-Length	消息体的长度, 以字节为单位
Last-Modified	资源上次修改的时间
Location	重定向(3xx)时使用, 客户方应该发送请求的新URL
Transfer-Encoding	消息体传输采用的编码(包括压缩) 方式, 比如gzip, chunked
Set-Cookie	设置Cookie, 可多次出现

HTTP GET和POST方法

如何传递表单(form)数据?

- 采用GET方法, 通过HTTP URL传递form数据
 - form数据作为URL最后面的查询字符串
 - 可能超过URL的限制, 且URL会被纪录到访问历史中

```
http://www.foo.com/myProg.cgi?surname=Mao&firstname=Dilin
```

- 采用POST方法, 查询字符串将通过消息体传递
 - POST方法后的URL为处理form的页面
 - Content-Length: 给出POST请求中的消息体长度
 - Content-Type: 给出消息体包含内容的类型, 比如text/plain, text/json等
 - application/x-www-form-urlencoded, 查询字符串采用%编码

```
<form action="http://www.foo.com/myProg.cgi" method="get">
  Surname: <input type="text" name="surname" >
  Firstname: <input type="text" name="firstname" >
  <button type="submit"> Send data </button>
</form>
```

Surname: Firstname:

```
GET /myProg.cgi?surname=Mao&firstname=Dilin HTTP/1.1
Host: www.foo.com
```

```
POST /myProg.cgi HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Host: www.foo.com
Content-Length: 27
```

```
surname=Mao&firstname=Dilin
```


块编码(chunked encoding)

客户方发送请求后如何知道返回的HTTP对象(消息体)的结束?

- 服务方发送响应后关闭TCP连接
 - Content-Length头部给出了消息体的长度
 - Transfer-Encoding头部给出了消息体是怎么编码的。如果为chunked, 表示采用块编码
 - 对象被分隔成多个块, 分多次传输, 一次一块
 - 每个块格式: chunk-size [chunk-ext]CRLF chunk-data CRLF
 - 第一行最前面是chunk-size:
 - 给出了chunk-data部分的长度
 - 以16进制方式描述
 - 如果chunk-size为0, 表示为最后一块
- 0<CR><LF><CR><LF>

HTTP/1.1 [302 Moved Temporarily](#)

Server: nginx

Date: Thu, 20 Oct 2016 00:41:27 GMT

Content-Type: text/html; charset=UTF-8

[Location: 2016/index.html](#)

Transfer-Encoding: chunked

Connection: keep-alive

A1

<!--[if lt ie 9]>

<script language="JavaScript" type="text/javascript">
 window.location.href="index.html";

</script>

<![endif]-->

0

维护 Client/Server 状态: Cookie

HTTP是无状态(stateless)的协议

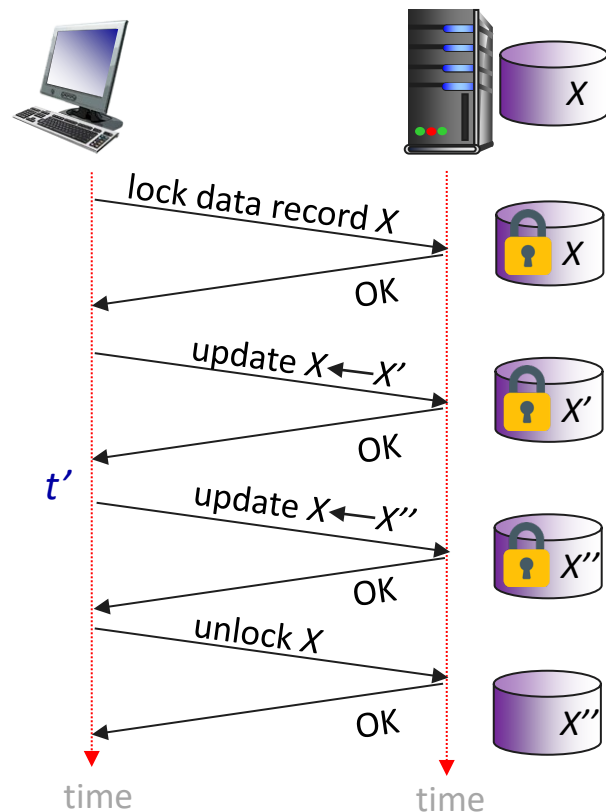
- HTTP服务器并不维护之前收到的**HTTP请求的状态**
- HTTP客户方也不需要维护状态
- HTTP请求是相互独立的
- 简化了服务器的设计，容易支持大量并发的HTTP请求
- HTTP协议提供**cookie机制来传递状态**信息：RFC 6265
 - HTTP响应：**set-cookie头部**设置cookie，cookie相当于状态ID
 - name=value
 - 可以包括多个Set-Cookie头部，每个set-cookie传递一个状态（比如会话ID和用户偏好等）
 - 本地(浏览器)的cookie文件：保存各个服务器返回的cookie
 - 后续的HTTP请求：**cookie头部**携带之前服务方返回的cookie
 - cookie头部携带了1个或者多个cookie，之间以分号隔开
 - Web站点的后台数据库：根据cookie找到相应的状态记录

Set-Cookie: SID=31d4dad42
Set-Cookie: lang=zh

Cookie: SID=31d4dad42; lang=zh

一个有状态协议的例子：

- 事务示例：客户方进行两次更新



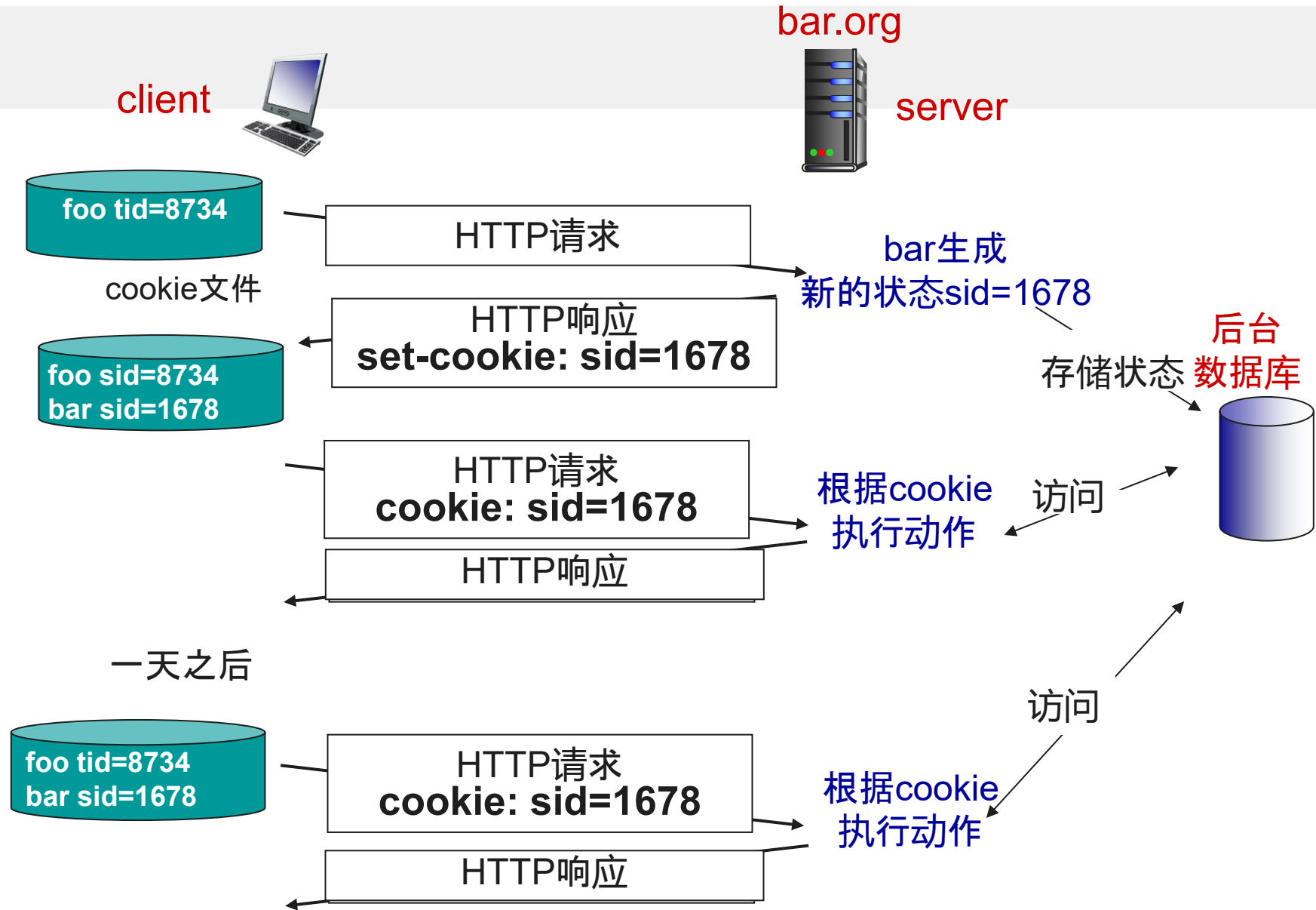
Q: 如果在t'，客户方崩溃或者网络断开？

HTTP: Cookie

- HTTP提供了传递Cookie的机制: HTTP请求的Cookie头部和HTTP响应的 Set-Cookie头部 (可多个)
- Web网站的后台服务器生成和处理Cookie
- 浏览器保存cookie
- 浏览器根据所访问的站点的主机名以及路径部分, 从保存的cookie文件中挑选出匹配的cookie, 然后通过Cookie头部传递

Cookie技术是把双刃剑

- 维护用户状态信息, 分析用户喜好, 向用户进行个性化推荐
- 跟踪用户网络浏览痕迹, 泄露用户隐私
- 用户可以限制Cookie的使用



HTTP: Cookie

- 每个Set-Cookie头部描述某个状态的值是什么，以及该状态的相关属性(可选)

- 格式 Set-Cookie: name=value; attr1=val1; attr2=val2
- 分号和冒号后面有一个空格

比如Set-Cookie: SID=31d4d96e407aad42; Path=/; Domain=example.com; Max-Age=1800

- Domain属性表示该cookie可适用的域名。缺省为仅当前服务器。如果指定，同时也包括其子域
 - Path属性表示该cookie可以用于的路径部分。缺省为请求URI的路径部分
 - **cookie-name+path+domain** 唯一标识一个cookie
 - Expires属性给出了cookie的过期时间，如果其为过去的日期，表示删除该cookie
 - Max-Age属性给出了cookie可以保存的最长时间，比Expires优先级高
 - 如果没有指定Expires和/或Max-Age属性，则该cookie仅仅用于当前浏览器会话
 - 其他属性还包括Secure, HttpOnly等
- Cookie头部包含了本地保存的Cookie中匹配的Cookie

Cookie: NAME1=VALUE1; NAME2=VALUE2

响应头部:

Set-Cookie: SID=31d4dad42; Path=/; Domain=example.com

Set-Cookie: lang=zh

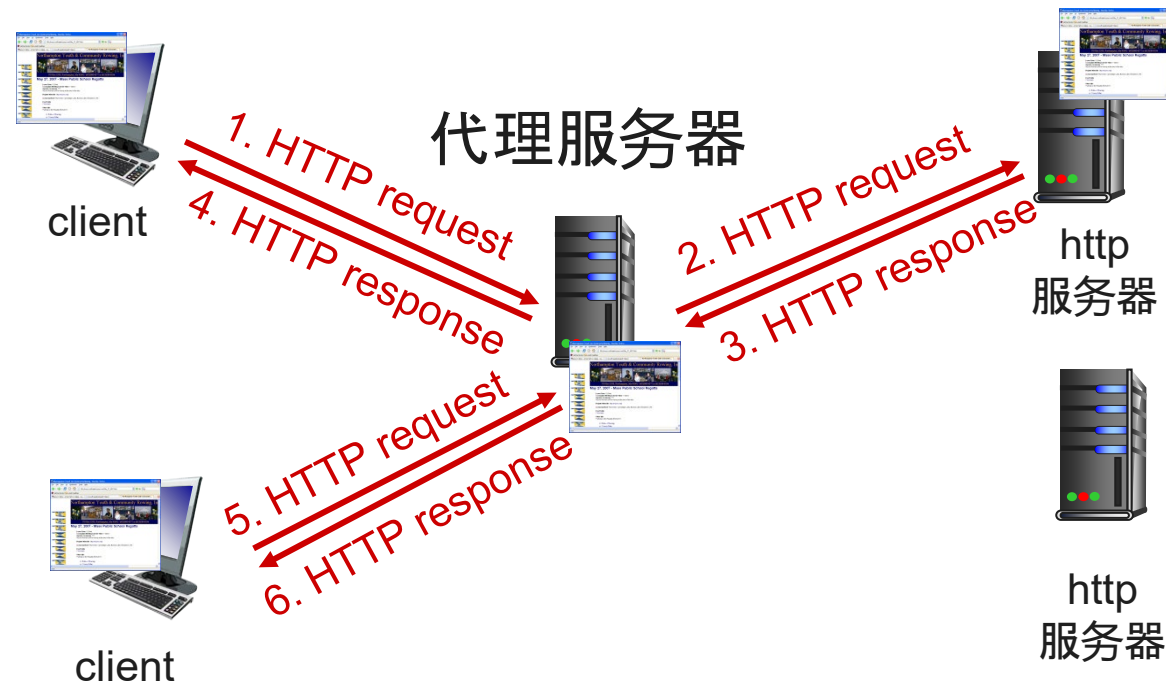
请求头部:

Cookie: SID= 31d4dad42; lang=zh

Web缓存

[RFC 9111] Web cache或Proxy Server: 代表初始服务器满足客户方的HTTP请求的网络实体

- 浏览器设置代理服务器的地址
- 用户浏览器访问HTTP服务器时, HTTP请求发送给代理服务器
 - 查找代理服务器的缓存, 如果没有命中:
 - 代理服务器作为HTTP协议的客户端方发送请求给服务器, 在收到资源后再发送HTTP响应给用户
 - 如果代理服务器的缓存中已有该资源, 则返回给用户
- 减少了响应时间
- 减少了外出链路的负载
- 整体上减少Internet上的Web流量
- 代理服务器充当了client方的HTTP服务器的角色
- 对于HTTP服务器而言, 代理服务器充当客户端的角色



```
GET http://mail.qq.com/ HTTP/1.1
Host: mail.qq.com
Proxy-Authorization: Basic aGVsbG86d29ybGQ=
User-Agent: curl/7.58.0
Accept: */*
Proxy-Connection: Keep-Alive
```

Web缓存示例

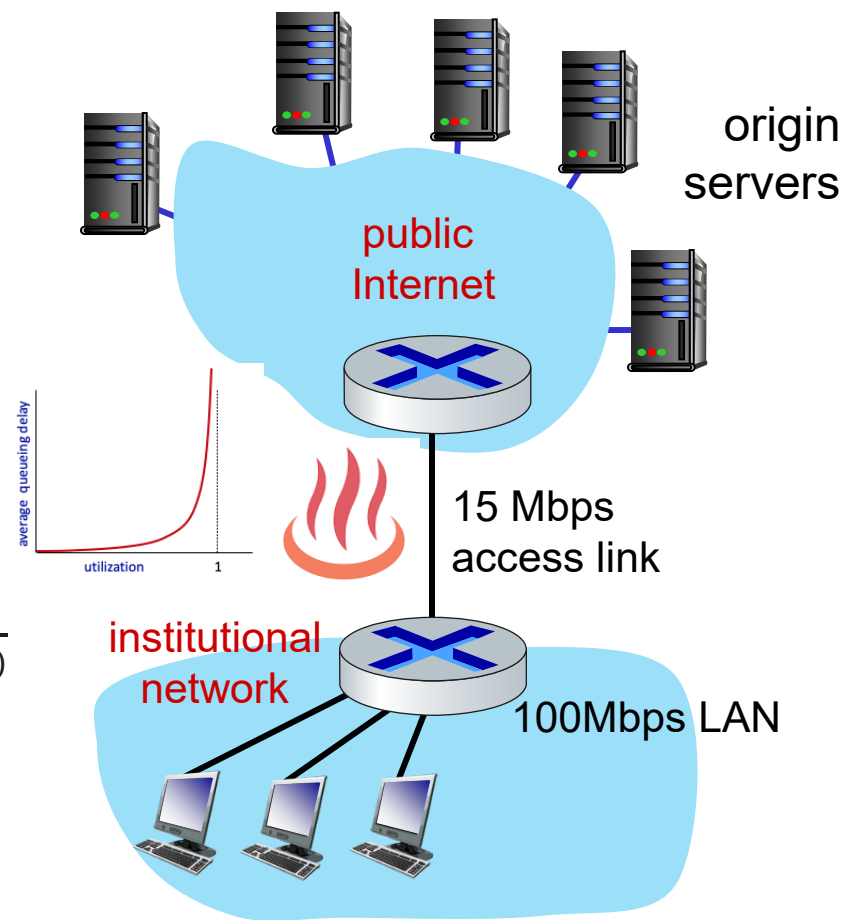
- 接入链路: 15Mbps
- 接入路由器到服务器的RTT(Internet Delay): 2sec
- Web对象: 1M bits
- 浏览器往服务器平均发送请求: 15请求/sec
 - 平均数据速率 = $15 * 1\text{M bits/sec} = 15\text{Mbps}$

性能分析:

- traffic intensity: $\rho = \text{数据速率} / \text{链路速率}$
- 100Mbps LAN上的 $\rho = 15/100 = 0.15$
- 15Mbps 接入链路的 $\rho = 15/15 = 1$ 平均排队延迟 $Q = \frac{1}{\mu} \frac{\rho}{(1-\rho)}$
- end-end delay = Internet delay + access link delay + LAN delay
(估计) = 2 sec + **minutes** + msecs

方法1: 升级接入链路: 15Mbps --> 100Mbps

- 100Mbps接入链路的 $\rho = 15/100 = 0.15$, 排队延迟10毫秒级别
- end-end delay = **2 sec** + msecs + msecs

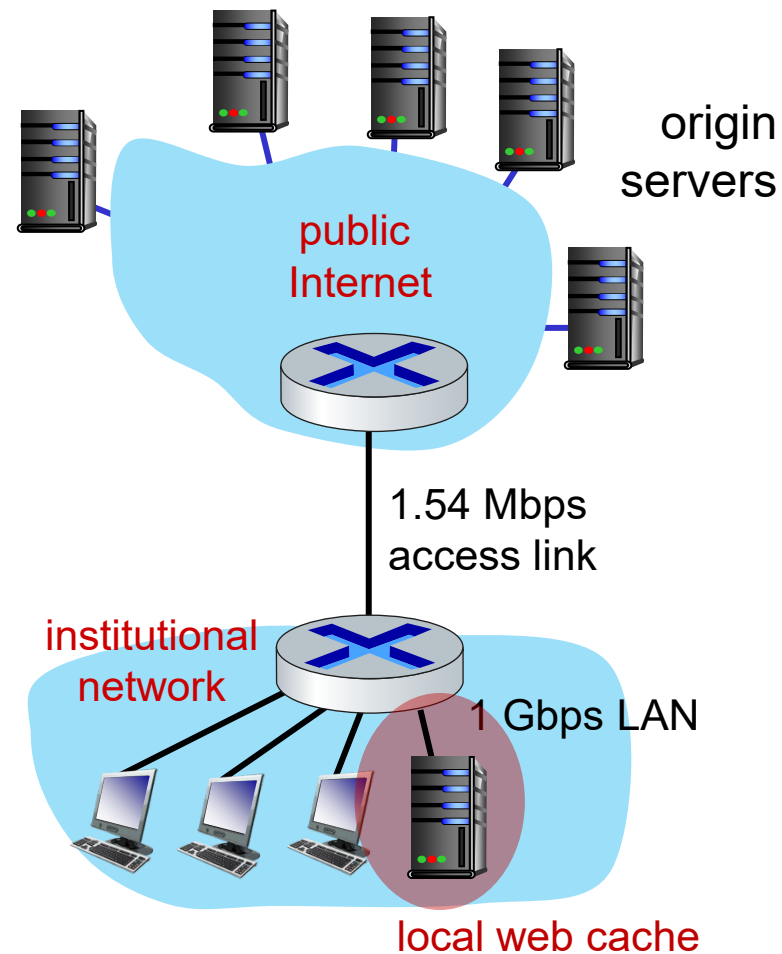


Web缓存示例

- 接入链路: 15Mbps
- 接入路由器到服务器的RTT(Internet Delay): 2sec
- Web对象: 1M bits
- 浏览器往服务器平均发送请求: 15请求/sec
 - 平均数据速率 = $15 * 1\text{M bits/sec} = 15\text{Mbps}$

方法2: 引入Web缓存, 相比升级链路的开销更小

- LAN上的流量强度 $\rho = ?$
- 接入链路的流量强度 $\rho = ?$
- end-end delay = Internet delay + access link delay + LAN delay
= ?



Web缓存：引入缓存后的延迟分析

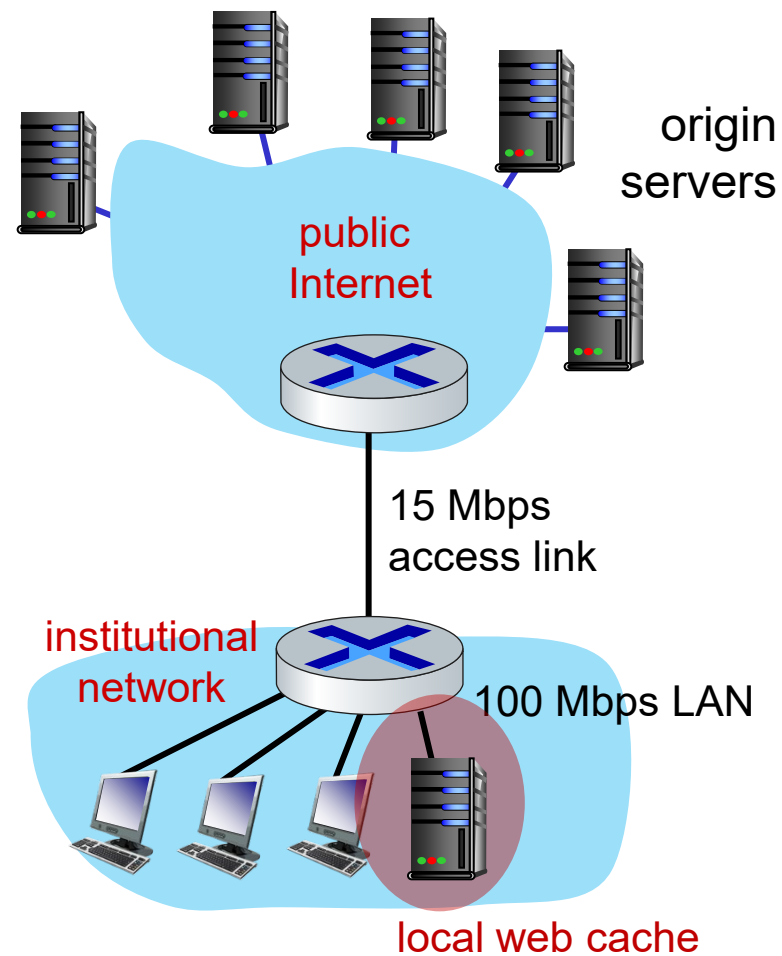
$$\text{end-end delay} = \text{Internet delay} + \text{access link delay} + \text{LAN delay}$$

缓存命中率一般为[0.2, 0.7]，假设hit rate = 0.4

- 接入链路：15Mbps
- 接入路由器到服务器的RTT(Internet Delay)：2sec
- Web对象：1M bits
- 浏览器往服务器平均发送请求：15请求/sec
 - 平均数据速率 = $15 * 1\text{M bits/sec} = 15\text{Mbps}$

方法2：引入Web缓存，相比升级链路的开销更小

- 40%的请求：LAN上的Web缓存给出响应
 - LAN上的平均速率 = $0.4 * 15 * 1\text{Mbit/sec} = 6\text{Mbps}$
 - LAN上的流量强度 $\rho = 6 / 100 = 0.06$
 - 排队延迟：大约10ms左右的延迟
- 60%的请求到达origin server
 - 接入链路上的平均速率 = $0.6 * 15 * 1\text{Mbit/sec} = 9\text{Mbps}$
 - 接入链路的流量强度 $\rho = 9 / 15 = 0.6$
 - 接入链路排队延迟 + LAN延迟：大约10ms左右延迟
 - 浏览器到服务器的延迟 $\approx 2\text{ secs} + 10\text{ms} = 2.01\text{ secs}$



- 方法2：平均延迟 = $0.4 * 0.01 + 0.6 * 2.01 \approx 1.2\text{ secs}$
- 相比升级接入链路的方法1(2秒)更低延迟, 开销更小

缓存控制

- 浏览器可以缓存（私有缓存）最近收到的HTTP响应（携带了Web对象）
- Web缓存（共享缓存）可以缓存HTTP响应
- 控制如何缓存： HTTP请求和**HTTP响应**的相关头部
 - Cache-Control: no-store 不允许缓存
 - Cache-Control: no-cache HTTP响应可缓存，但使用时必须验证
 - Cache-Control: max-age=delta HTTP响应的最大age为多少秒，过期后应该验证
 - HTTP响应的freshness_lifetime = max-age 或 Expires - Date
 - HTTP响应的current_age： 在缓存中的时间 + HTTP响应在网络中传输的时间（可根据发送HTTP请求和收到HTTP响应的时间间隔估计）
 - HTTP响应是否仍然fresh?

$current_age \leq freshness_lifetime$

实时或隐私内容
Cache-Control: no-store

动态内容
Cache-Control: no-cache

静态内容
Cache-Control: max-age=3600

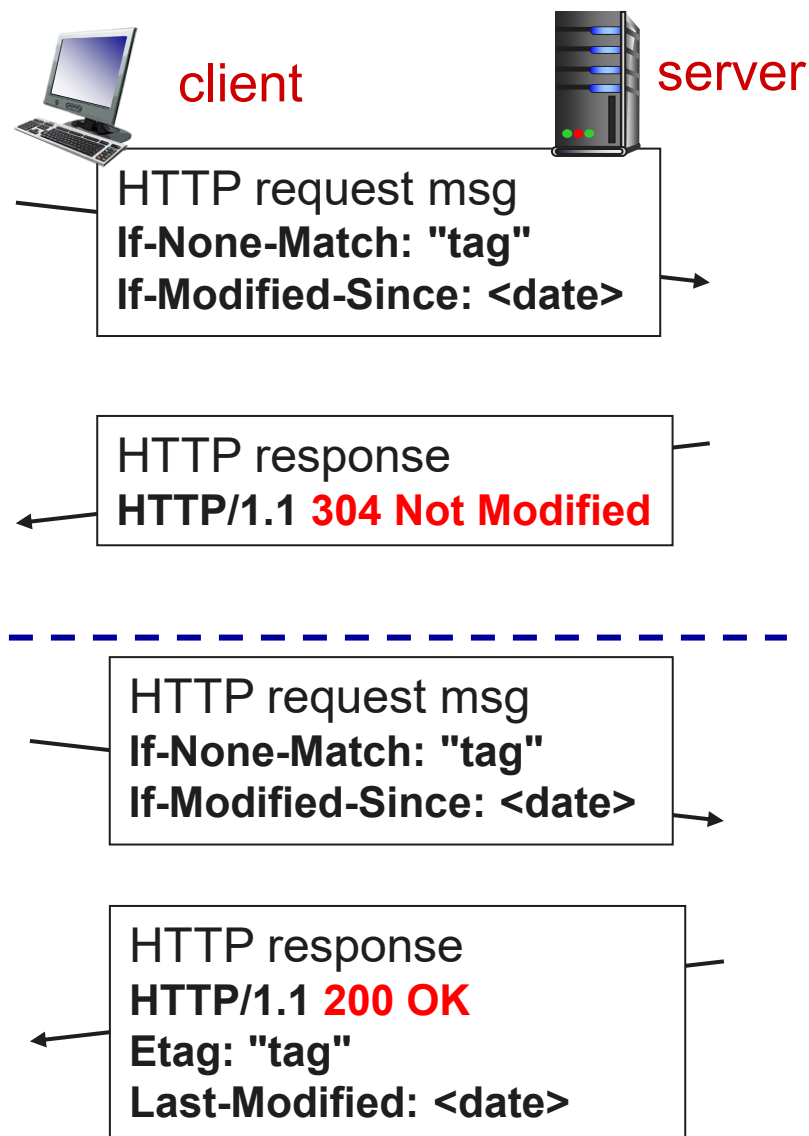
头部	描述
Expires	资源过期时刻
Date	原始服务器产生HTTP响应的时刻
Age	估计的HTTP响应的生存时间
Cache-Control	可指定是否缓存，是否要验证缓存仍然最新，最大age多少等

条件GET

如何验证缓存的对象是否有效（新鲜）？

- HTTP响应中的验证有效性的相关头部
 - Etag: 资源的唯一标识, 如果改变了, 说明资源有更新
 - 可以是版本控制系统的版本号
 - 可以是资源内容+元数据的哈希值
 - Last-Modified: 服务器返回资源时, 该资源最后一次被修改的时间
- 发送条件GET检查缓存中的资源对应的原始资源是否有更新
 - If-None-Match: 散列值改变时返回新资源
 - If-Modified-Since: 在此之后修改过时返回新的资源。If-None-Match头部优先
 - 其他条件GET头部还包括: If-Match, If-Unmodified-Since和If-Range等

HTTP响应头部	描述
Last-modified	资源上次修改的时间
Etag	唯一标识资源内容的Tag



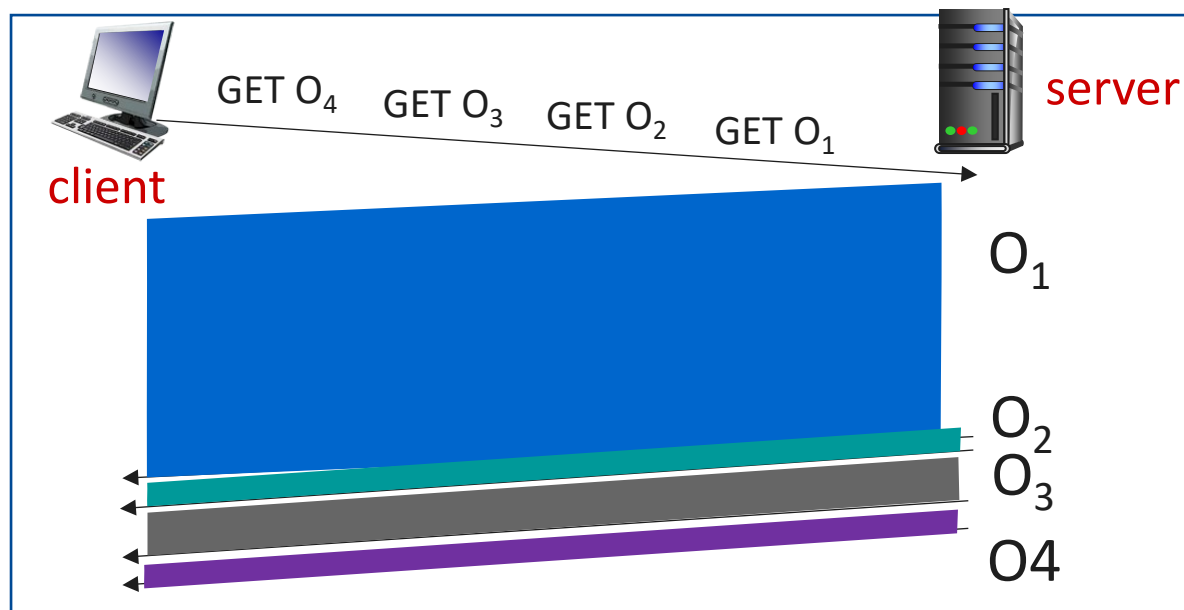
HTTP/1.1的队头阻塞问题

考虑访问一个包含1个视频对象和3个小对象的页面，采用HTTP/1.1: 管道化的持续连接方式

- 发送多个HTTP请求，响应按照请求到来的先后顺序回应
- 小的对象要等待前面的大对象传输完成后才可以发送，即出现队头阻塞(Head-of-Line Blocking)
- 如果TCP段丢失，需要进行重传，使得后续的对象传输延迟

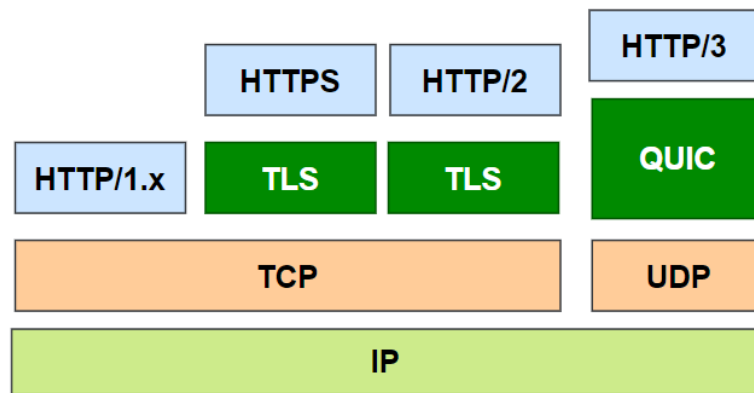
==> 浏览器的实现中，采用多个（一般限制为6个）并行的TCP连接

- 通过“欺骗”获得更多的带宽
- 减少页面呈现延迟



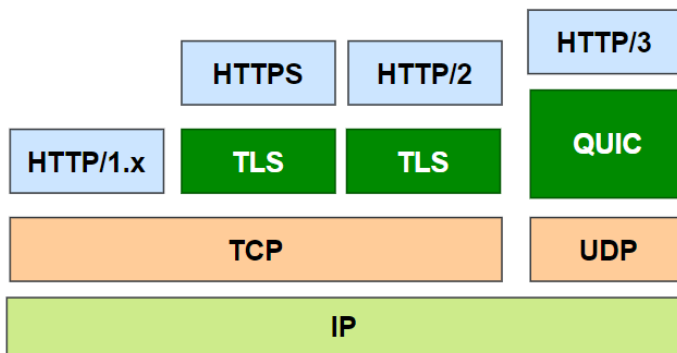
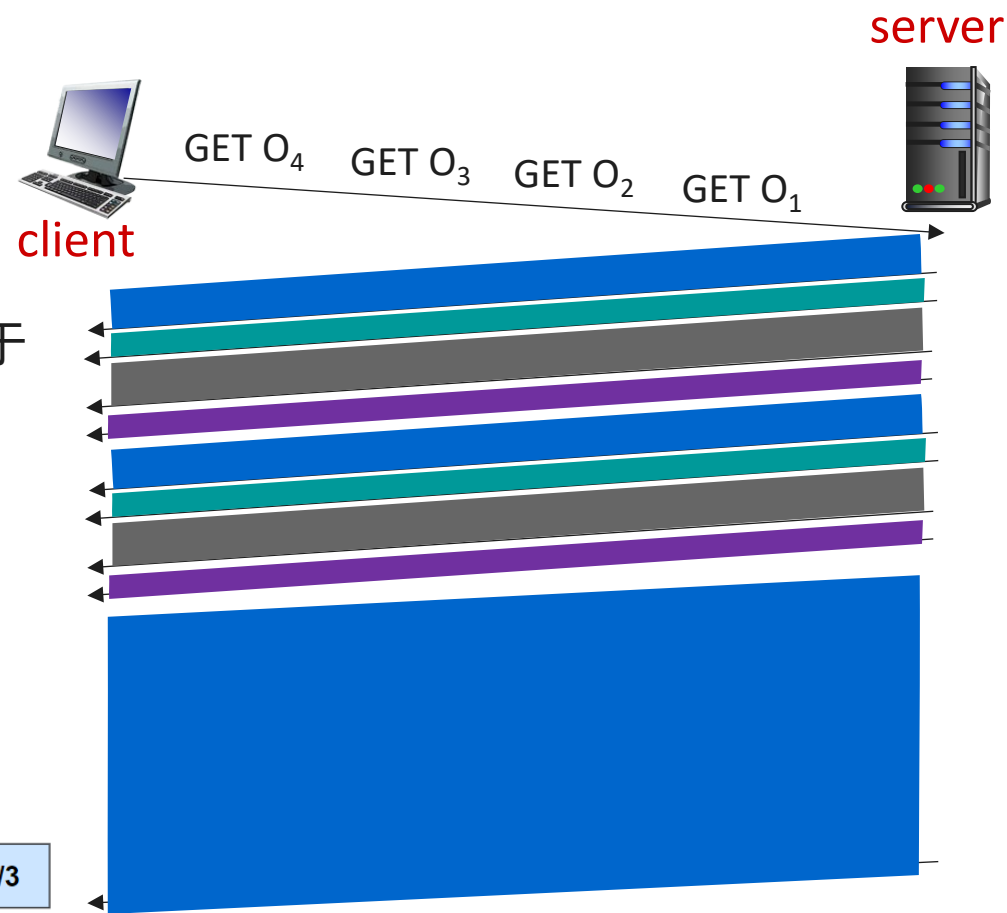
HTTP/2

- 2015年RFC7540定义了HTTP/2。最新版本为2022年6月发布的RFC9113
- 关键目标：提高带宽利用率，减少延迟
- 仍使用原有的HTTP框架，包括请求方法、状态码、请求和响应头部等，与HTTP/1.1并不兼容
- **新的封装和传输方法：**
 - 不再采用基于文本方式的请求和响应协议，而是基于**二进制方式的封装**，实现起来更加高效
 - 允许多个Web对象**多路复用**
- 引入**头部压缩**机制，减少头部传输的开销
- 页面中的对象的重要性不同，引入HTTP请求**优先级机制**，优先级高的请求所请求的对象优先传输
- 支持**服务方推送(push)**：服务方将客户方尚未请求但有很大可能即将请求的对象主动推送给客户方



HTTP/2

- HTTP请求和HTTP响应属于同一个流(STREAM)
- HTTP响应携带的资源可以分解成多个帧 (Frame)
- 属于多个流的帧可以多路复用在一起, 通过同一条TCP连接传输
 - 每个流的帧的顺序得到保证
 - 多个流的帧可以交错(interleaved)复用在一起。如何复用依赖于优先级和具体的实现
- 平均延迟减少:
 - 小的对象相比HTTP/1.1更早到达
 - 大的对象稍微延迟到达
- HTTP/2没有解决TCP段丢失重传使得后续对象的传输延迟的问题
- HTTP/3:
 - 安全
 - 连接管理
 - 流量控制, 差错控制, 拥塞控制



主要内容

2.1 网络应用原理

2.2 Web和HTTP

2.3 电子邮件

2.4 DNS

2.5 P2P文件分发

2.6 视频流和内容分发网

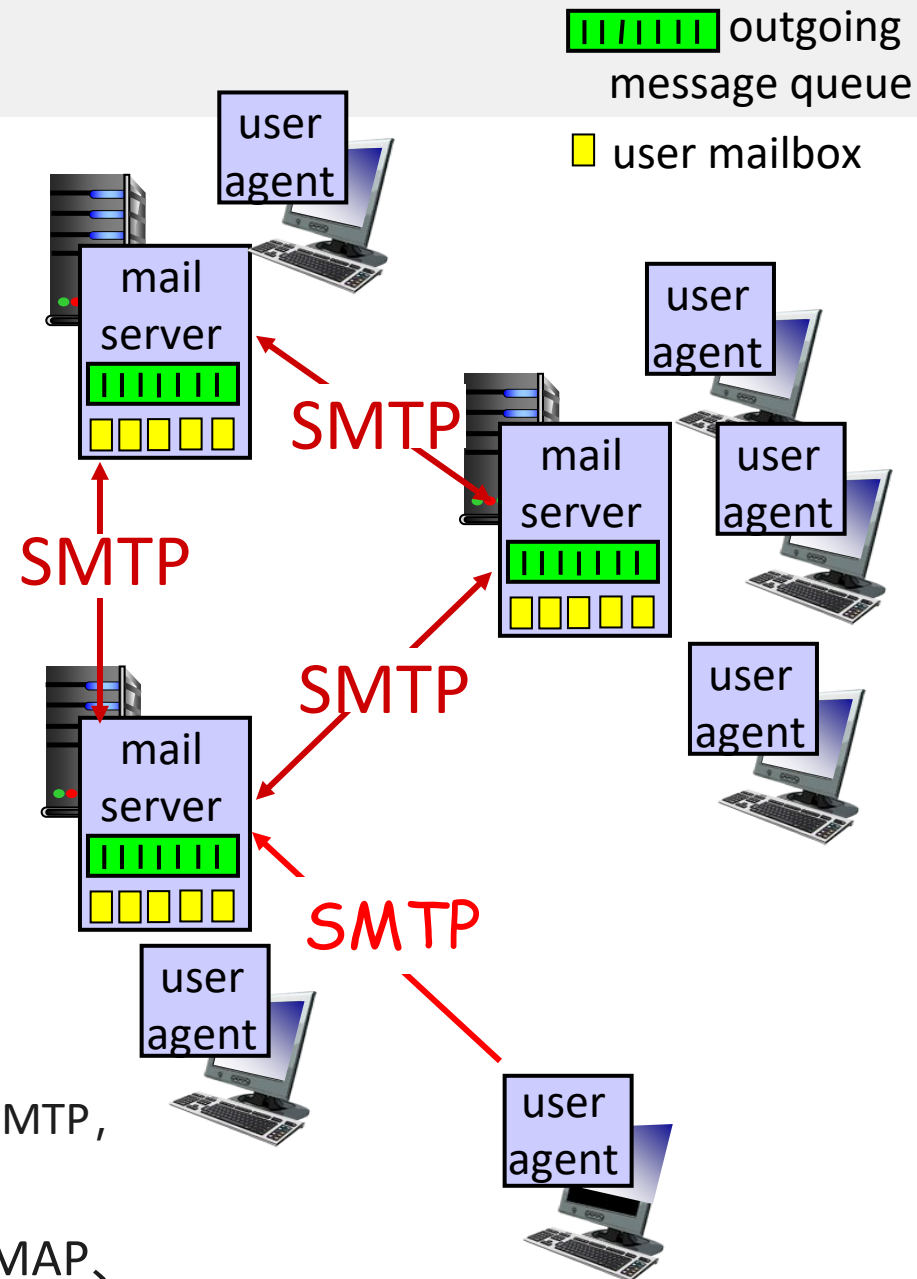
2.7 套接字编程

E-mail(Electronic mail)

- 电子邮件是最早的TCP/IP应用之一
- E-mail地址格式: user@domain
 - user为本地解释, 可能大小写相关
 - domain: 查询DNS找到邮件服务器, 大小写无关

四个主要部件

- **用户代理(Mail User Agent)**: 即“电子邮件应用程序”
 - 撰写、阅读邮件
 - Outlook, Thunderbird, 基于Web或者智能手机上的邮件应用
- **邮件服务器(Mail Transfer Agent)**:
 - 邮箱: 送达给(本地)用户的到来邮件
 - 邮件队列: 要发送给其他用户的邮件
- **SMTP协议**: **邮件服务器之间**传递邮件
 - **邮件提交(mail submission)协议**: 用户代理到邮件服务器, 基于SMTP, 一般要求发送方认证
- **邮件访问协议**: 获得邮件服务器上的邮箱中的邮件, POP3、IMAP、Web Email等



SMTP(Simple Mail Transfer Protocol)

- RFC 821/2821/5321给出了SMTP协议
- 基于C/S模型，建立在可靠的数据传输协议TCP之上，端口号25
- SMTP是一种Push协议
 - 邮件从用户逐步推送经过多个邮件服务器(MTA, Mail Transfer Agent)，直到到达最终用户所在的邮件服务器
 - 大部分邮件途中一般只会经过1个或2个邮件服务器，即发送方所在的邮件服务器以及接收方所在的邮件服务器
 - 每一跳采用SMTP协议可靠递交邮件



- 基于**7-bit ASCII文本**的请求-响应协议：
 - Client发送请求: EHLO、MAIL FROM、RCPT TO、DATA、QUIT等
 - Server发送响应: 状态码 (3个数字) + 文字说明，之间以空格或-隔开
 - SMTP响应一般为一行，但也可支持多行：
 - 每行以状态码-开始，后面是文字
 - 最后一行以状态码开始，后面是空格，再后面为文字

```
C: EHLO ubuntu
S: 250-mail
  250-AUTH LOGIN PLAIN
  250 OK
```


SMTP命令

命令 (大小写无关)	说明
HELO domain	打招呼, 给出发送者的domain或IP地址
EHLO domain	替代HELO, 采用ESMTP(Extended SMTP)。其响应给出了所支持的SMTP扩展
MAIL FROM:<addr>	发送者邮件地址。注意冒号前后没有空格
RCPT TO:<addr>	接收方邮件地址, 可发送多个RCPT TO命令
DATA	接下来是邮件部分, 直到只有一个. (dot)的行结束, 即CRLF.CRLF出现。如果邮件部分的某行的行首字符为点, 则替代以两个点。接收方发现行首为点且该行后面还有字符, 则去掉第一个点
QUIT	退出

状态码	描述
2xx	成功
3xx	成功, 期待后续命令。比如收到DATA请求时的状态码
4xx	临时拒绝
5xx	永久拒绝

Envelope: MAIL/RCPT命令
Content: DATA命令

```
C: EHLO ubuntu
S: 250-mail
    250-AUTH LOGIN PLAIN
    250 OK
C: MAIL FROM:<xiucao@fudan.edu.cn>
S: 250 xiucao@fudan.edu.cn... Sender ok
C: RCPT TO:<wang@pku.edu.cn>
S: 250 wang@pku.edu.cn... Recipient ok
C: RCPT TO:<zhang@tsinghua.edu.cn>
S: 250 zhang@tsinghua.edu.cn... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Blah blah blah.....
C: .....
C: .
S: 250 Ok
C: QUIT
S: 221 fudan.edu.cn closing connection
```

服务器收到CRLF.CRLF后表示邮件接收完成, 发送250 Ok

- 客户方收到250 OK, 表示邮件发送成功
- 客户方没有收到250 OK, 但连接断开: 假设收到451响应, 即会认为邮件发送不成功, 以后会再次发送, 从而可能出现重复邮件

- 邮件信封：MAIL FROM和RCPT TO命令
 - RCPT TO的地址决定了该邮件最终投递的用户邮箱
 - MAIL FROM的地址：在邮件递交出现问题时发送通知(Bounce)邮件到该地址
 - 邮件离开SMTP环境时，可在邮件内容最前面添加**Return-Path头部**，保存MAIL-FROM地址
 - Bounce邮件的MAIL FROM地址为 <>，即不会对该邮件再发送Bounce邮件
- 邮件内容：Data命令
 - SMTP协议本身并不要求Data部分必须严格按照电子邮件格式，它可以是普通的ASCII文本
 - MTA的邮件服务器配置可能对于Data部分有相应的要求
- 途中每个邮件服务器都会在邮件内容最开始添加一个**Received头部**，打上邮戳
 - 可了解邮件经过的路径
 - 避免回路（如果Received头部超过100个）

- Received头部的基本格式：by和datetime必须
Received: from <sending host>
by <receiving host>
with <mail protocol>
id <msg-id>
for <mail-address>;
<datetime received>

Return-Path: <dlmao@fudan.edu.cn>
Received: from barracuda.fudan.edu.cn ([2001:da8:8001:2:225:90ff:fed8:bb36])
by mx.google.com with ESMTP id
ih4si31738720pab.37.2023.10.18.17.51.42
for <dilin.mao@gmail.com>;
Tue, 18 Oct 2023 17:51:42 -0700 (PDT)
Received-SPF: pass (google.com: domain of dlmao@fudan.edu.cn designates
2001:da8:8001:2:225:90ff:fed8:bb36 as permitted sender) client-
ip=2001:da8:8001:2:225:90ff:fed8:bb36;
Received: from fudan.edu.cn ([61.129.42.33]) by barracuda.fudan.edu.cn with
ESMTP id RS3ftwxAzlsG4KD7 for <dilin.mao@gmail.com>; Wed, 19 Oct 2023
08:51:40 +0800 (CST)
Received: by ajax-webmail-app3 (Coremail) ; Wed, 19 Oct 2023 08:49:30 +0800
(GMT+08:00)
Date: Wed, 19 Oct 2023 08:49:30 +0800 (GMT+08:00)
From: "Mao Dilin" <dlmao@fudan.edu.cn>
To: dilin.mao@gmail.com
Subject: hello from dlmao
Content-Type: multipart/mixed;
boundary="-----=_Part_366479_1777208066.1476838170514"
MIME-Version: 1.0
Message-ID: <28a53653.1ac5e.157da6a1f92.Coremail.dlmao@fudan.edu.cn>

-----=_Part_366479_1777208066.1476838170514
Content-Type: text/plain; charset=UTF-8
Content-Transfer-Encoding: 7bit

Dear Dilin,

This is a test message, please ignore it.

Mao Dilin

-----=_Part_366479_1777208066.1476838170514
Content-Type: text/plain; name="dummy.txt"
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="dummy.txt"

YWJjZGVmZ2hpamtsbW4=
-----=_Part_366479_1777208066.1476838170514
Content-Type: image/png; name="project5.png"
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="project5.png"

iVBORw0KGgoAAAANSUhEUgAAAVYAAAA9CAIAAAEOjuZvAAAAAXNSR0
IArs4c6QAAAAARnQU1BAACx
jwv8YQUAAAAJcEhZcwAADsMAAA7DAcdvqGQAABDWSURBVHhe7Z1/T
FX1G8fdYqPFgjRwiuScqUFz
-----=_Part_366479_1777208066.1476838170514--

邮件格式

- RFC 822/2822/**5322**定义，由US-ASCII字符组成（1到127）
- 多行文本，行分隔符为CRLF
- 每行长度（不包括CRLF）最长998，建议78

头部： 7-bit US-ASCII 可打印字符(ASCII码32-126)

- 一个头部一般对应一行，由name和value组成，中间以冒号:分隔
- 注意**冒号前**不能有空格
- 头部名大小写无关

邮件体： 7-bit US- ASCII 字符（ASCII码1-127）

- 头部和邮件体之间以1个空行分隔 <CRLF><CRLF> **\r\n\r\n**
- 邮件内容没有任何其他限制

头部中的邮件地址：

- 单个邮件地址： user@domain
<user@domain>
"Full Name" <user@domain>
- 多个邮件地址： 以逗号分割

```
From: John Doe <jdoe@machine.example>  
To: Mary Smith <mary@example.net>  
Cc: <tom@foo.com>, bob@foo.com  
Subject: Saying Hello  
Date: Fri, 21 Nov 2023 09:55:06 -0600  
Message-ID: <1234@local.machine.example>
```

```
This is a message just to say hello.  
So, "Hello".
```

<header name>: <head value>

如果一行太长时，可以多行，后面的行以**空格或Tab**开始

<header name>: <header value part1>

<white-space> <header value part2>

<white-space><header value part3>

邮件头部

空行

邮件体

邮件格式：常见头部

M: Mandatory
O: Optional

N: Normally present
S: should be present

类别	头部名	必须/可选	出现次数	描述
来源日期	Date	M	1	邮件撰写完毕准备发送的时间
来源	From	M	1	邮件的源(一个或多个发送者)
	Sender	O	1	邮件的实际发送者
	Reply-To	O	1	在回信时应该回送给谁(一或多个地址)。如果没有，则来自于From头部
目的	To	N	1	邮件的接收者，可以多个，之间以逗号隔开
	Cc	O	1	邮件的(抄送)接收者，可以多个
	Bcc	O	1	邮件的(密送)接收者，该头部在其他接收者处被移走
消息ID	Message-ID:	S	1	唯一标识该邮件，在注入邮件系统时生成 <xxx@xxx>
	In-Reply-To:	O	1	在回信时一般会包括，指出所回信件的ID(一个或多个)
	References	O	1	包含了多封信件的ID，用于关联多个邮件，将其组织为邮件会话
信息	Subject	N	1	信件主题
跟踪	Received:	MTA	无限制	类似邮戳功能，纪录经过的服务器信息以及相应的时间
	Return-Path	MTA	1	信件最后递交（离开SMTP环境）时记录通过MAIL FROM传递的反向路径

- 第1跳邮件服务器一般会检验邮件头部的合法性，自动添加Date，Message-ID头部(如果邮件没有这些头部) 等
- 用户(邮件服务器)自定义的头部，一般以x-开头

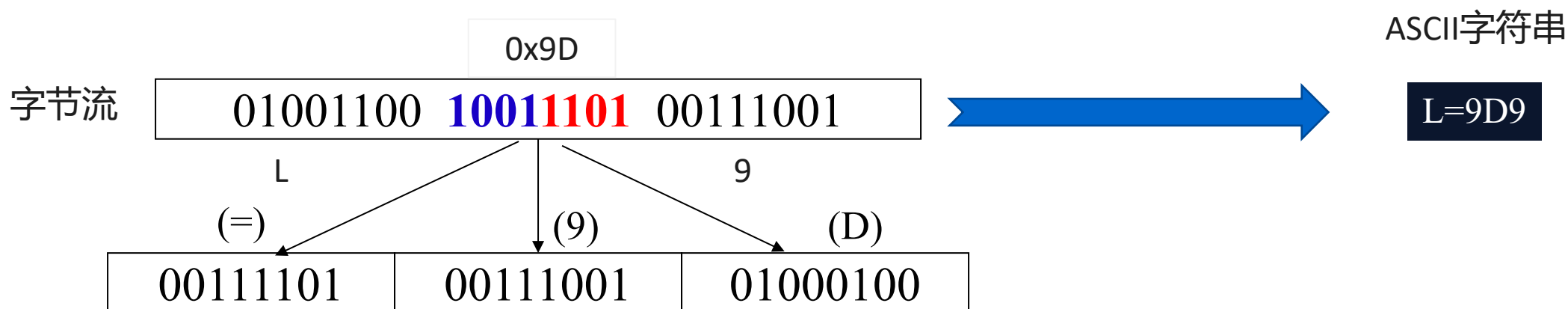
MIME(Multipurpose Internet Mail Extension)

- RFC822邮件格式的问题：
 - 要求7-bit ASCII字符，非英文文本邮件可能出现问題
 - 无法发送附件（一般为二进制格式）
- RFC 2045, 2046, 2047引入了MIME：
 - 仍然采用原有的RFC 822邮件格式，仍然采用原有的邮件传输(SMTP)和访问协议
 - 传输编码：支持非ASCII字节串与7-bit ASCII文本之间的转换
 - 内容类型/子类型： 可将**多个附件、消息文本整合**在一起
 - MIME实体(entity)：由头部以及消息体组成的一个相对完整的部分
 - 通过MIME头部描述MIME实体的类型和编码等信息
 - MIME实体携带的可以是文本信息、图片、音频、视频、应用支持文档（比如docx, pdf等）
 - MIME实体也可以是由多个MIME实体组成，从而允许将多个部分的内容有机地组合在一起成为一个邮件

传输编码: quoted-printable

非ASCII字节串与7-bit ASCII文本之间的转换

- quoted-printable: 引用可打印编码, 编码为可打印字符 + 回车换行
- 用于只有少量的非ASCII可打印字符的情况
 - 可打印字符: ASCII码从32到126, 共95个字符, 包括空格、数字、大小写英文字母和标点符号
- 可打印字符不需要转换
- 回车换行符(\r\n)不需要转换
- 其他字符应该进行转换: `'={:02X}'.format(octet)`
 - 1个字节(octet)转换成3个字符: "=" + 两个十六进制 (大写) 数字 (对应该字节的值)
 - 等于号(=, ASCII码0011 1101)转换为3D。

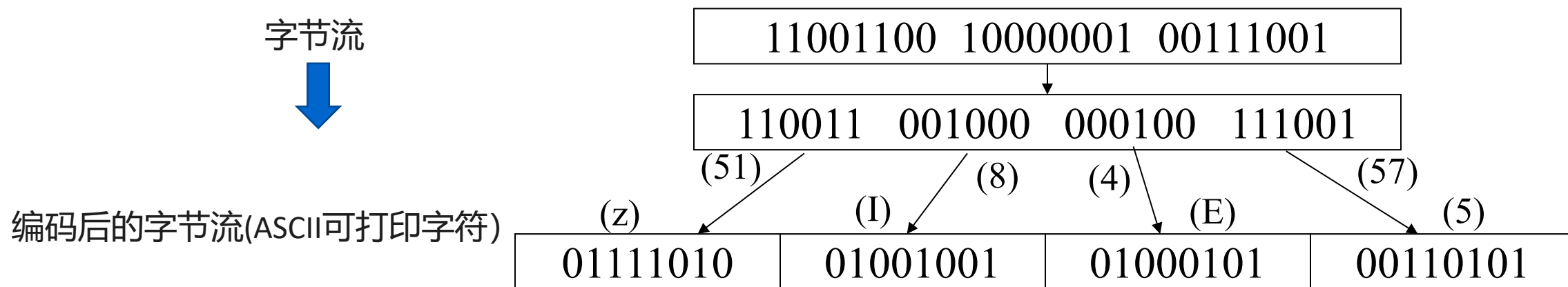


传输编码: Base64

$$\text{编码效率} = \frac{\text{有效的数据}}{\text{实际传输的数据}} = \frac{3}{4}$$

非ASCII字节串与7-bit ASCII文本之间的转换

- 基本思想: 要编码的内容以3个字节为单位, 转换为4个ASCII可打印字符
- 3个字节(24个比特)以**每6比特为一组**, 共4组
 - 每组(6比特)对应的64个值依次用字符A~Z、a~z、0~9、+和/表示
- 以3个字节 (24比特) 为整体编码时, 最后可能只剩下一个字节或两个字节
 - 在后面 (右边)填上足够的比特0, 凑够6比特, 再转换为ASCII可打印字符
 - 最后再**添加多个等号**: 剩下1个字节时添加2个等号, 2个字节时添加1个等号
- 上述编码规则只用了65个不同字符(包括用于填充的=), 根据需要可以自由地 (每76字符) 添加CRLF



最后剩1个字节: 001110**01**

→ 001110 01**0000**

→ 0Q → 0Q==

最后剩2个字节: 10000001 0011**1001**

→ 100000 010011 1001**00**

→ gTk → gTk=

MIME 头部

Content-Type: text/plain; charset=UTF-8

Content-Type: image/jpeg; name="fudan.jpg"

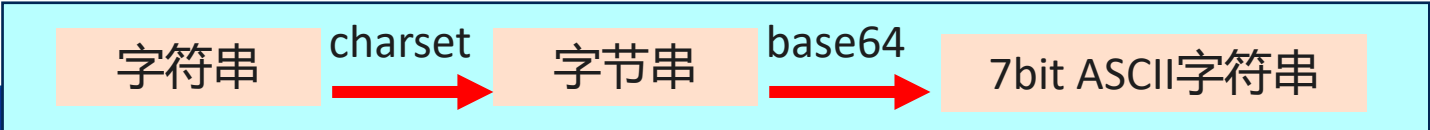
- MIME头部允许有（多个）参数部分，以分号开始，格式为 ;name1=value1;name2=value2

主要头部	描述
MIME-Version	版本号，目前1.0，只在整个消息头部中出现
Content-Type	内容的类型， type/subtype, 缺省text/plain; charset=us-ascii
Content-Transfer-Encoding	编码方式，可选，缺省为7bit，还可以是8bit, base64, quoted-printable等
Content-Disposition	RFC 2183, 对内容(附件)怎样处理。inline表示内嵌显示，而attachment表示另外保存，如 Content-Disposition: attachment; filename="http.pptx"

- Content-Type: type/subtype

type	subtype	type	subtype
text	plain,html,enriched,css	application	octet-stream, zip,pdf,msword, vnd.ms-powerpoint...
image	jpeg, png, gif	multipart	mixed, alternative, parallel, digest, related, 由多个entity组成
audio	basic,mp3	message	rfc822, partial, external-body, 封装了一个消息
video	mpeg,mp4		

- 类型为text：一般包含参数charset，说明文本属于哪个字符集(编码方式) 中的字符，缺省us-ascii



MIME: 类型multipart

(主要) 子类型

- mixed:各个独立的子部分组合在一起, 比如附件
- alternative:同一内容的不同表示, 比如文本和HTML版本

参数boundary给出了各个子部分的边界

- 分界行: **两个连字符开始, 然后是boundary参数给出的字符串, 最后可包含空格 (可选)**
- Boundary参数随机选择, 保证MIME Entity中不会出现某行等于分界行, 或者某行以分界行的内容开始
- multipart类型的body部分的结束: **两个连字符开始, 然后是boundary参数给出的字符串, 最后是两个连字符**

```
Mime-Version: 1.0
Content-Type: multipart/mixed;
    boundary=="001_Dragon614750641803_=="
```

```
--==001_Dragon614750641803_==
Content-Type: text/plain;charset="gb2312"
Content-Transfer-Encoding: base64
```

```
1eLKx9K7t+Ky4srU0.....
```

```
--==001_Dragon614750641803_==
Content-Type: application/octet-stream; name="test.doc"
Content-Transfer-Encoding: base64
```

```
UEsDBBQABgAIAAAAIQ.....
```

```
--==001_Dragon614750641803_==--
```

SMTP与HTTP

- 都采用基于ASCII文本的命令/响应交互，状态码描述命令执行的情况
- HTTP： 客户方pull
- SMTP： 客户方push
- HTTP： 一个HTTP响应消息封装了一个Web对象
- SMTP： 一个邮件可以封装一个对象，一个Multipart邮件封装了多个对象
- SMTP： 采用持续连接
- HTTP/1.0采用非持续连接， HTTP/1.1引入持续连接
- SMTP是有状态的协议
- HTTP是无状态的协议
- SMTP要求消息（头部+消息体）都是7-bit ASCII字符
- SMTP服务器通过CRLF.CRLF表示邮件的结束

邮件访问协议

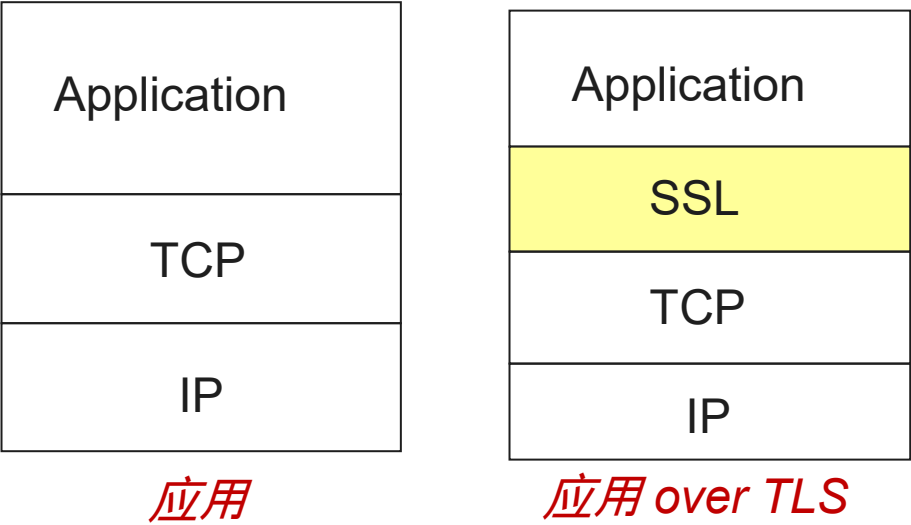
- 邮件提交：
 - 基于SMTP协议
 - 本地命令行：如果User Agent就在邮件服务器上
 - Web-Email：基于Web，由Web服务器提交



- 邮件访问：
 - 本地命令行：如果User Agent就在邮件服务器上
 - Web-Email：基于Web，访问邮件服务器上的邮箱
 - 基于邮件访问协议
 - POP3(Post Office Protocol Version 3)：将邮件下载到本地，在本地组织管理
 - IMAP(Internet Mail Access Protocol)：邮件在服务方存储，在服务方管理，通过文件夹组织
 - 允许不同主机对于**邮件有统一的访问体验**
 - 尽管邮件会在服务方存储，客户方一般也会缓存那些已经下载阅读的邮件

众所周知的端口号

协议	常规端口号	implicit app over TLS	explicit app over TLS
HTTP	80		443
SMTP	25	587 (邮件提交)	465
POP3	110	110	995
IMAP	143	143	993



主要内容

2.1 网络应用原理

2.2 Web和HTTP

2.3 电子邮件

2.4 DNS

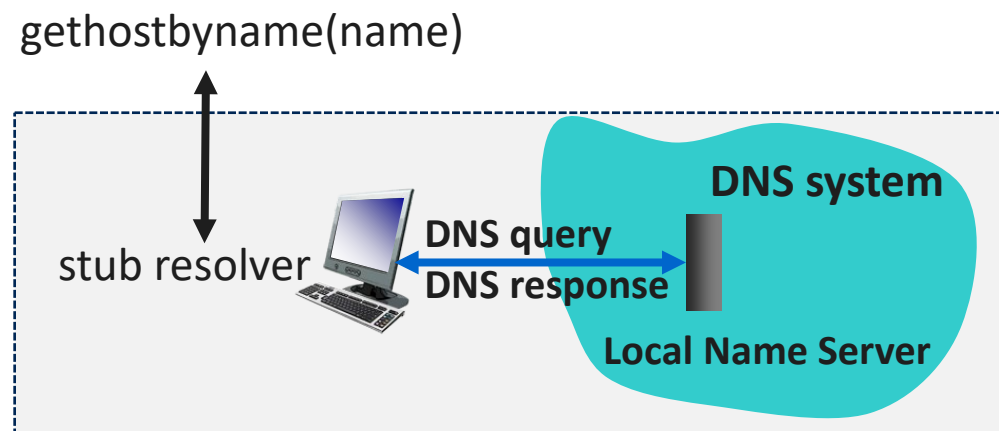
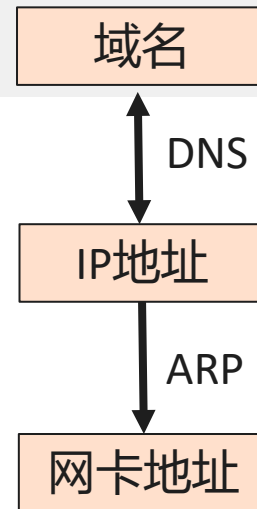
2.5 P2P文件分发

2.6 视频流和内容分发网

2.7 套接字编程

DNS (Domain Name System) 域名系统

- Internet的名字和地址：
 - 网卡地址(MAC地址)：标识某一块网卡
 - IP地址：ID + Locator, **固定长度**的32比特整数，标识主机，也包含该主机的位置（所在的网络）
 - 主机名字（域名）：**可变长**的方便记忆的一系列字符，没有地理界限(位置)的概念
- DNS: [1987年 RFC 1034/1035]
 - 域名与某种类型的值之间的映射**
 - 类型为A，值为域名所对应的IP地址
 - 类型为AAAA，值为域名所对应的IPv6地址
 - 保存了域名与值映射的**分布式数据库**
 - 域名如何组织？域名以树的形式组织
 - 各个DNS服务器如何管理和维护域名树？
 - 如何查询该分布式数据库以解析域名？应用层协议
 - 主机、本地（解析）域名服务器、（某些）域名服务器之间的DNS消息交互
 - 关键的Internet基础设施：在端系统实现
 - 复杂性位于网络的“边缘”的一个范例



查看配置：
windows: ipconfig/all
MacOS: scutil --dns

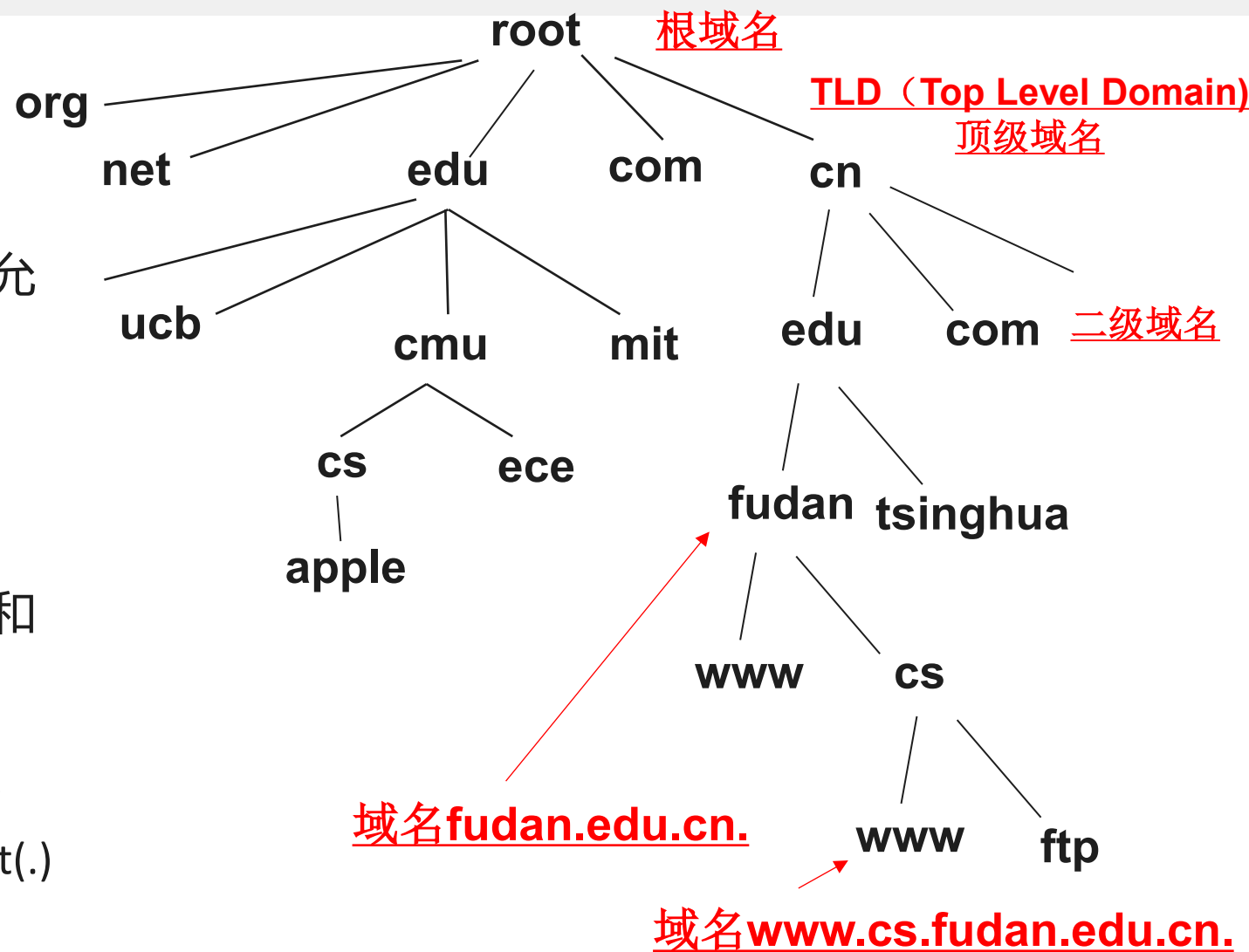
```
# /etc/resolv.conf
nameserver 202.120.224.26
search fudan.edu.cn
```

DNS 历史：从主机文件→层次域名空间

- 平面(Flat)名字空间和集中管理：
 - (Inter)NIC(Network Information Center)维护一个 "hosts.txt" 文件，记录名字与IP地址的映射
 - 主机通过FTP协议下载到本地 /etc/hosts.txt
- 平面名字，命名空间不足
- 集中管理：
 - 单点故障 (Single Point of Failure)
 - 通信容量 (Traffic Volume)
 - 远距离的集中式数据库 (Distant Centralized Database)
 - 维护(Maintenance)
- 没有可扩展能力
 - Akamai DNS Servers: 2.2T DNS Queries/day
- 层次(hierarchical)名字空间
 - 采用树的方式组织，方便、易扩展
 - 只要保证节点下面的子节点的名字不同就可以了
- 巨大的分布式数据库：上亿记录，每条记录简单(name, type, value)
- 处理大量的（每天上万亿）查询请求
 - 相比写(更新)而言，更多的是读(查询)
 - 毫秒级的响应速度：几乎所有应用都要用到DNS
- 组织和物理上的非集中式：
 - 上亿的机构负责管理和维护域名记录
- 较好的可靠性和效率
- 不需要原子性和强一致性：允许域名映射有暂时不一致的情况

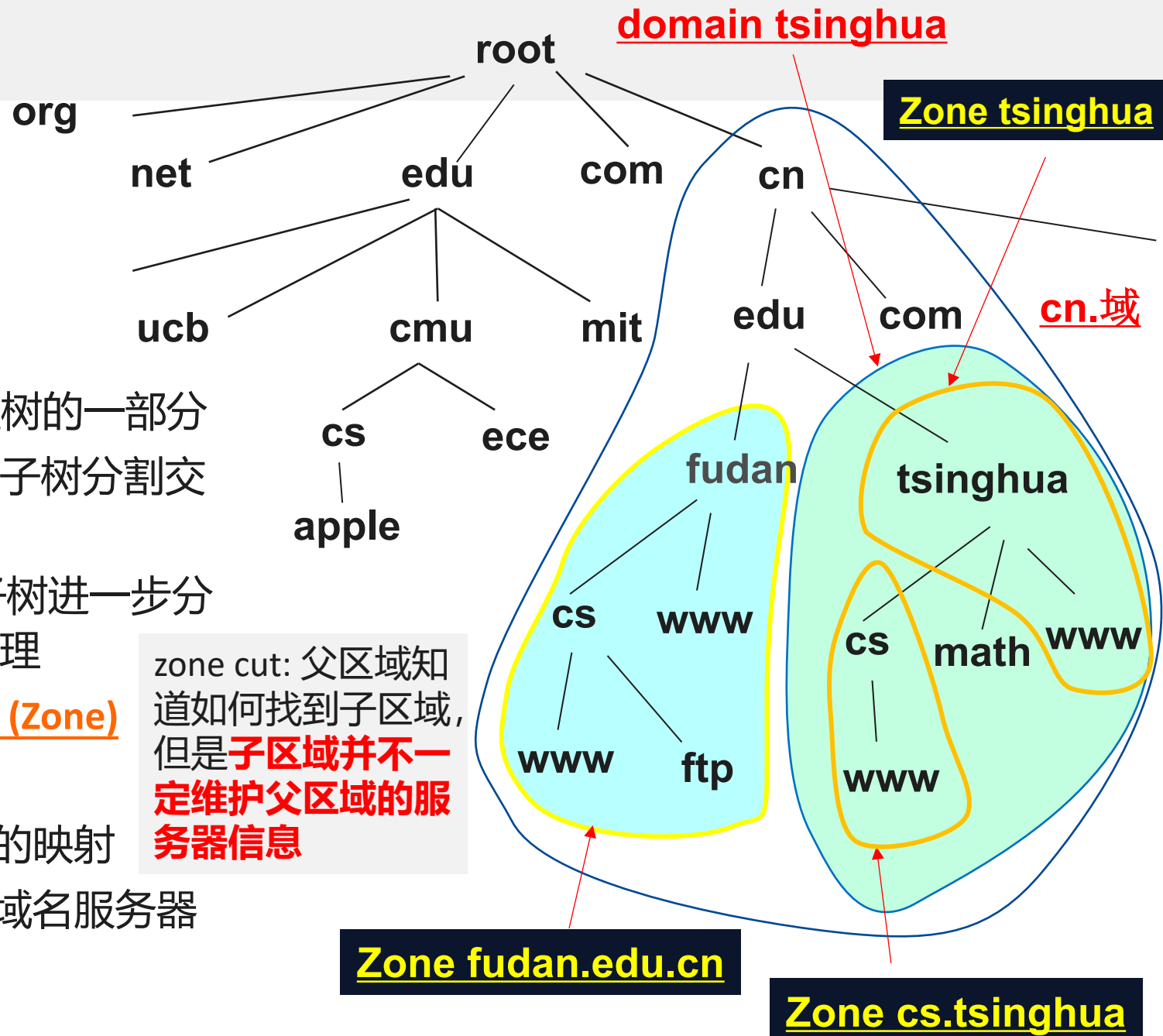
域名层次结构

- 域名树最多128层，第0层为root
- 标签(Label):树中的每个节点有一个label
 - 最多63个字节，**大小写无关**
 - 英文字母、数字和连字符，连字符不允许出现在首尾
 - root: 空字符串（空标签）
- 域名：每个节点有一个域名
 - 域名最长255个字节
 - 域名定义的不是地理界限，而是组织和管理界限
- 绝对域名FQDN（Fully Qualified Domain Name）由该节点的label开始一直到root为止的一系列label组成，label之间中间以dot(.)分隔



域名的维护：域名服务器

- 域Domain: domain fudan.edu.cn
 - 域名树的一颗**完整子树**
 - 域的名字为该子树的**根节点的域名**
- 整棵域名树采用分布式方式管理
 - 权威(authoritative) DNS服务器**管理树的一部分
 - 根域名服务器**将整棵树下的第一层子树分割交给不同的(TLD)DNS服务器管理
 - 顶级域名(TLD)服务器**将其管理的子树进一步分割授权给更下一层的**域名服务器**管理
 - 域名服务器管理的基本单位为**区域 (Zone)**
 - 修剪的子树**
 - 由域名服务器管理区域中名字的映射
 - 子树的某些分支可授权给其他域名服务器管理



根域名服务器

根域名服务器(Root Server): 有关根域名服务器的信息可以访问 www.root-servers.org

- 负责管理根区域，**根区域的权威域名服务器**
- 采用DNSSEC提供安全性
- 根区域将下面的所有子树授权给下一级的TLD服务器
- 根区域的更新由ICANN (Internet Corporation for Assigned Names and Numbers)协调管理，ICANN也负责管理根服务器l.root-servers.net
- 13个根服务器 {a-m}.root-servers.net，分布在不同的国家，由12个不同的组织维护
- 根服务器在全球还有许多Mirrors，目前总共有1988个instance
 - 采用IP Anycast技术：与根服务器有相同的IP地址，通过BGP路由协议实现
 - DNS请求会路由到最近的根服务器或者根服务器的镜像
- 每个域名服务器：
 - 一般都知道**根服务器的名字和IP地址的映射**(root.hints, named.ca等文件)
 - 解析域名时，如果不知道负责管理域名的域名服务器时，可询问**官方最后联系渠道**（根服务器）
- 根域名服务器平均每天有约100 billion(十亿)次查询，来源于 <https://rssac002.root-servers.org/>

.	3600000	NS	A.ROOT-SERVERS.NET.
A.ROOT-SERVERS.NET.	3600000	A	198.41.0.4
A.ROOT-SERVERS.NET.	3600000	AAAA	2001:503:ba3e::2:30
.	3600000	NS	B.ROOT-SERVERS.NET.
B.ROOT-SERVERS.NET.	3600000	A	170.247.170.2
B.ROOT-SERVERS.NET.	3600000	AAAA	2001:1b8:10::b

Top-Level Domain 顶级域名

- 通用域 "generic domains" (gTLD): 表示域名的类别

Label	描述	Label	描述
com	商业组织	aero	航空
edu	教育组织	biz	商业或公司
org	非盈利组织	coop	合作组织
net	网络提供者	info	信息服务提供者
mil	军队	museum	博物馆
gov	政府	name	个人
int	国际组织	pro	职业组织

根和顶级域名等信息可访问:

<https://www.iana.org/domains>

中国互联网信息中心CNNIC, China Internet Network Information Center
负责管理.cn域名

<https://www.cnnic.cn>

- 国家域 "country code"(ccTLD): 表示国家和地区的代码
 - 一般两个字母, 比如uk、us、de、fr和cn等
 - 第二级进一步可采取类别和地区的方式来描述, 比如 edu.cn和sh.cn
- arpa (address and routing parameter area): 为Internet基础设施服务, <https://www.iana.org/domains/arpa>
 - in-addr.arpa用于反向域名解析 (如IP地址→域名)
- 目前有超过1500个顶级域名

权威域名服务器

- 一个区域一般建议至少由两个以上域名服务器管理
 - 区域的规模越大，部署的域名服务器也越多
- 考虑到负载均衡和提高可靠性
- 主域名服务器(Primary Name Server)：维护区域的域名映射
- 从域名服务器(Secondary Name Server)：从主服务器获得映射的拷贝 (replica)
- 主和从域名服务器都是该区域的**权威服务器 (authoritative server)**，拥有最权威和最新的映射
- 一个DNS服务器可以充当多个区域的主或者从服务器

区域信息：资源记录(Resource Record)

- 区域的主服务器负责维护区域信息，由多条资源记录组成
 - 资源记录RR: (**name**, **ttl**, **class**, **type**, **value**)
 - 类(Class): 用于Internet时取值为IN
 - TTL(Time to Live): 该记录在缓存时的有效期(秒), 86400表示1天
 - DNS响应以RRset(资源记录集) 的形式描述, 由name/class/type相同, 但是value不同的RR组成
- **Type=SOA**
 - name: 域(区域)
 - value: start of authority
 - 区域的第一个RR
 - 关于区域的相关信息(主域名服务器和联系人) 以及**主从服务器之间**保证区域信息同步的相关信息

```
$ dig fudan.edu.cn soa
;; ANSWER SECTION:
fudan.edu.cn.      3600  IN   SOA  ns.fudan.edu.cn. root.ns.fudan.edu.cn. 2000000547 18000 14400 3600000 86400
```

区域信息：资源记录(Resource Record)

- **Type=NS**

- **name:** 域(区域)
- **value:** 管理该域的权威服务器

- **type=A(或者AAAA)**

- **name:** 主机名
- **value:** IPv4地址(或IPv6地址)

- **Type=CNAME** “canonical name”

- **name:** 别名
- **value:** 正式的名字

- **Type=MX**, 无MX记录时使用A/AAAA

- **name:** 域
- **value:** 管理该域中用户电子邮件的服务器, 由优先级(更小数字意味更高优先级) 和服务器名组成

- Type=PTR, TXT, SRV等

```
$ dig fudan.edu.cn ns
```

fudan.edu.cn.	600	IN	NS	ns.fudan.edu.cn.
fudan.edu.cn.	600	IN	NS	ns2.fudan.edu.cn.
fudan.edu.cn.	600	IN	NS	ns1.fudan.edu.cn.

```
$ dig www.fudan.edu.cn
```

www.fudan.edu.cn.	237	IN	A	202.120.224.81
www.fudan.edu.cn.	237	IN	A	202.120.224.129

```
$ dig www.fudan.edu.cn aaaa
```

www.fudan.edu.cn.	600	IN	AAAA	2001:da8:8001:2::129
www.fudan.edu.cn.	600	IN	AAAA	2001:da8:8001:2::81

```
$ dig www.baidu.com
```

www.baidu.com.	301	IN	CNAME	www.a.shifen.com.
www.a.shifen.com.	76	IN	CNAME	www.wshifen.com.
www.wshifen.com.	112	IN	A	103.235.46.39

```
$ dig fudan.edu.cn mx
```

fudan.edu.cn.	600	IN	MX	10 mailrelay2.fudan.edu.cn.
fudan.edu.cn.	600	IN	MX	1 mx-fudan-edu-cn.icoremail.net.
fudan.edu.cn.	600	IN	MX	10 mailrelay1.fudan.edu.cn.

区域信息: Zone文件

- SOA RR: 第一个RRA, 且只有一个SOA: 主域名服务器、联系人以及主从域名服务器之间如何进行区域传输等
- NS RR:
 - 描述被授权管理该区域的域名服务器
 - 如果本区域有子区域, 则:
 - 通过NS RR 将子区域授权给另一个域名服务器管理
 - 管理子区域的域名服务器的域名属于子区域时需要 **glue record**
 - 父区域的NS和glue record都是non-authoritative
 - 子区域中的NS和glue record为authoritative
- 多个A或AAAA RR: 描述域名和IP地址(IPv6地址) 的映射
- MX RR: 描述给该区域中的用户发送电子邮件时应该发给哪个邮件服务器
 - 如果无MX RR, 域名所对应的IP地址就是邮件服务器的地址
- DNS服务器管理的区域信息的维护:
 - 管理员人工更新 (修改区域文件), 正在运行的DNS服务器重新读取区域文件
 - **DDNS(Dynamic DNS)**: RFC2136/3007, 允许通过DNS消息交互来更新名字映射, 即支持动态域名

对于区域edu.cn, 这两条记录都是 non-authoritative



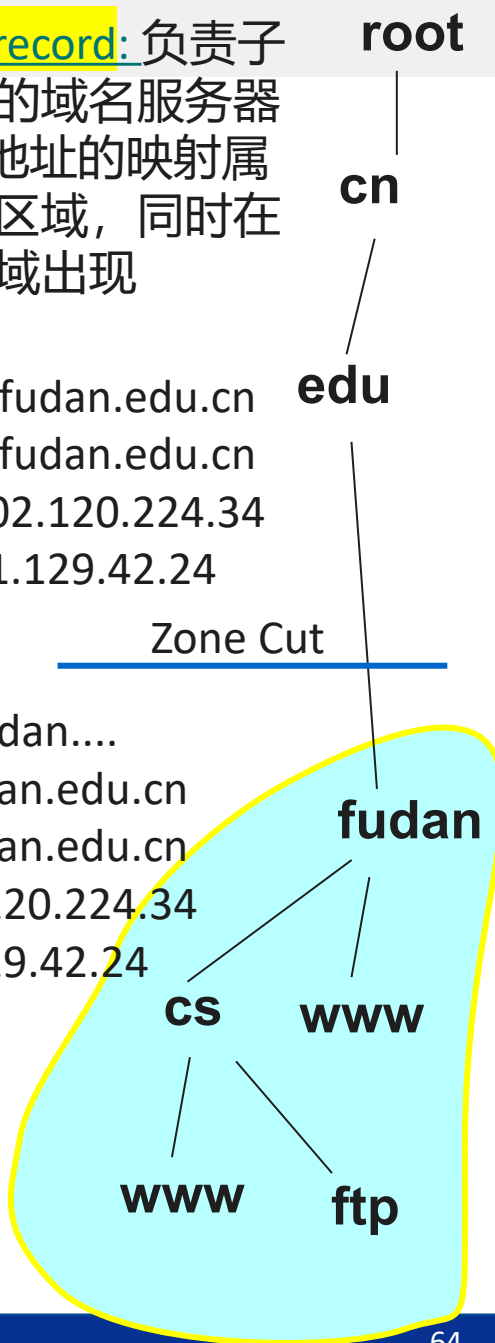
zone edu.cn:

```
fudan.edu.cn. NS ns1.fudan.edu.cn
fudan.edu.cn. NS ns2.fudan.edu.cn
ns1.fudan.edu.cn. A 202.120.224.34
ns3.fudan.edu.cn. A 61.129.42.24
```

zone fudan.edu.cn:

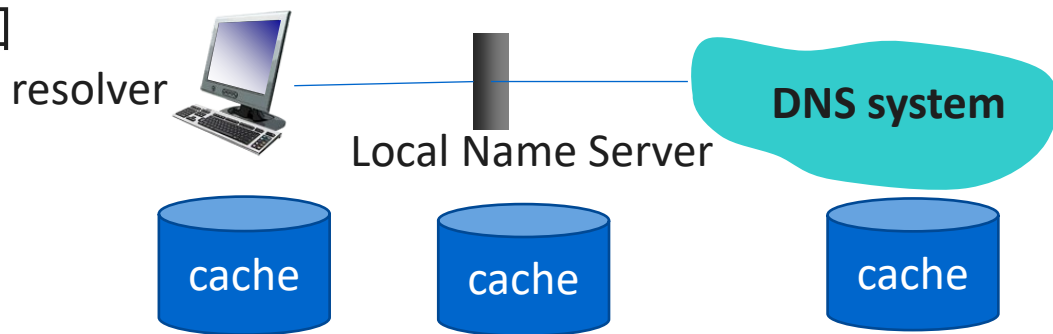
```
fudan.edu.cn. SOA ns.fudan....
fudan.edu.cn. NS ns1.fudan.edu.cn
fudan.edu.cn. NS ns3.fudan.edu.cn
ns1.fudan.edu.cn. A 202.120.224.34
ns3.fudan.edu.cn. A 61.129.42.24
```

glue record: 负责子区域的域名服务器与IP地址的映射属于子区域, 同时在父区域出现



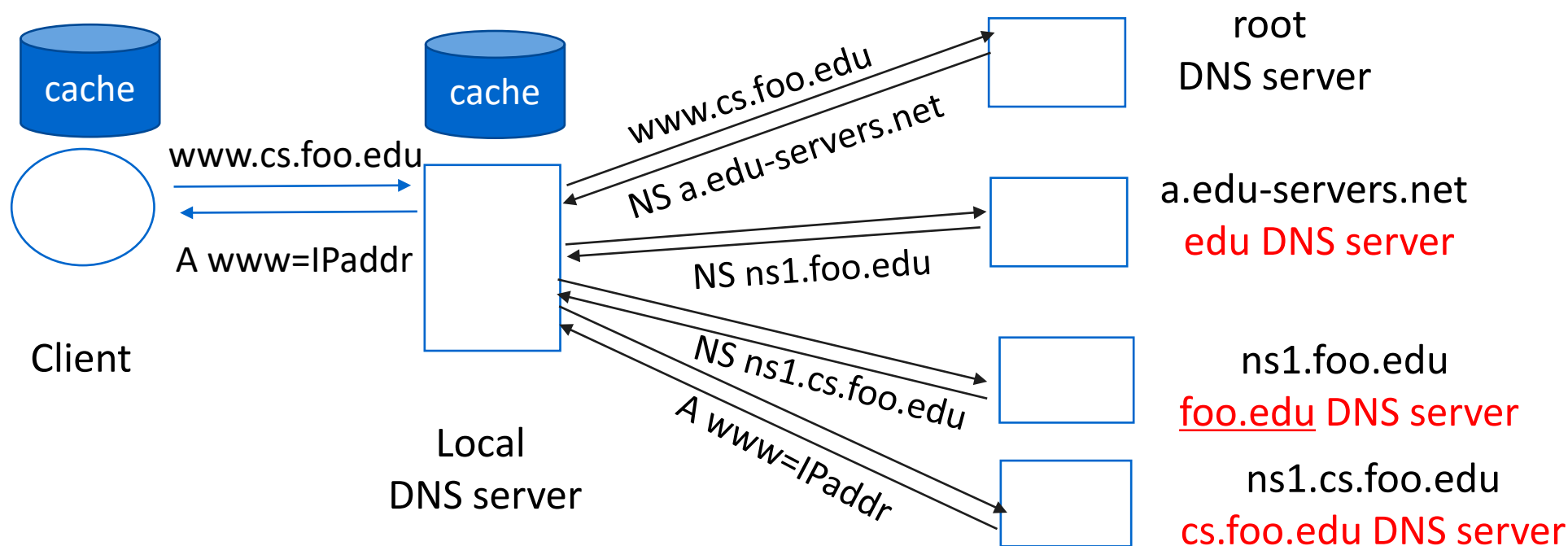
域名解析

- 主机一般会配置本地域名服务器，也称为**解析服务器**，为用户（主机）提供DNS解析服务
- 主机的应用程序访问(stub) resolver解析库，通过解析服务器访问DNS提供的服务
- 解析服务器严格来说不属于域名层次中的一部分
- 主机与本地域名服务器之间一般采用递归查询，本地域名服务器充当proxy功能，将DNS查询请求转发到维护域名空间的域名服务器，在得到解析结果后将其发送给主机
- 解析过程中的主机和解析服务器一般会将解析结果缓存
- 域名服务器在负责管理域名树中的一部分(即管理某些区域)的同时，一般也可充当解析服务器
- 每个ISP（包括大公司、学校）至少有一个解析服务器
- 目前Internet也有许多(6万多) 开放的解析服务器，比如
 - google提供的8.8.8.8和8.8.4.4
 - Level 3 Communications提供的4.2.2.{1-6}
 - OpenDNS提供的208.67.222.222和208.67.220.220
 - 阿里提供的223.5.5.5和223.6.6.6
 - 腾讯云DNS： 119.29.29.29



域名解析过程

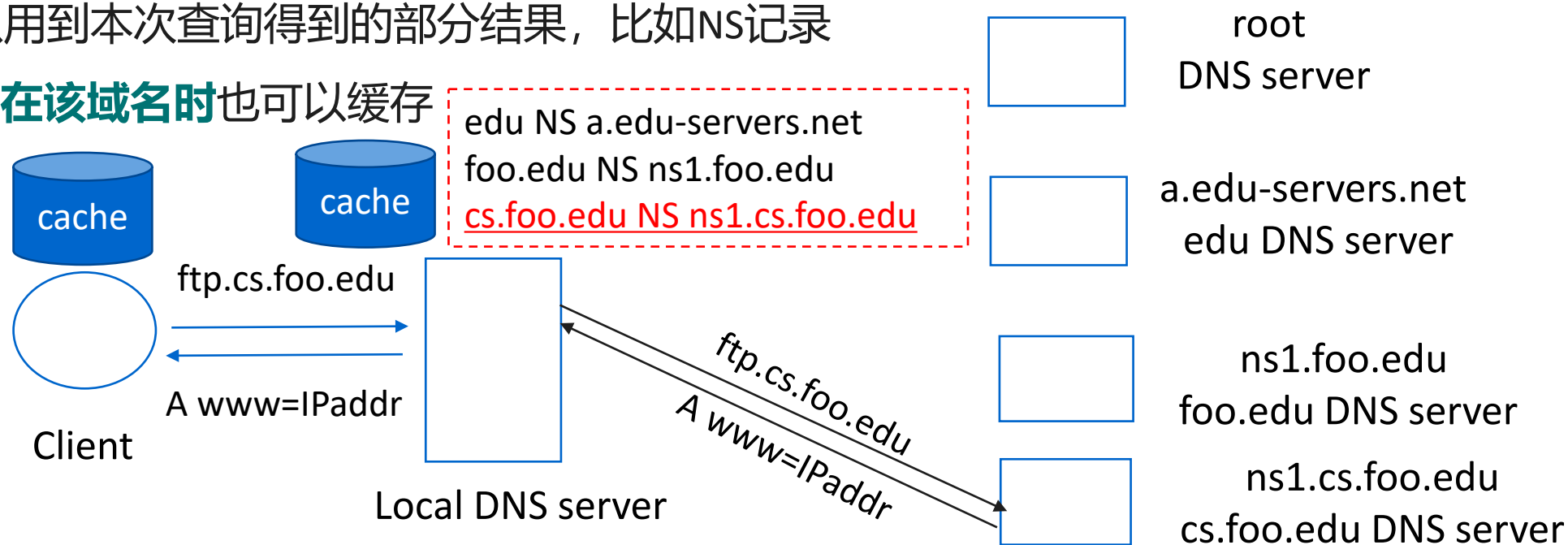
- 每次请求-应答过程中返回的是解析结果还是查找解析结果的指引(referral)
 - 递归(recursive)方式：发送请求给下一个服务器，对方给出解析结果
 - 迭代(iterative)方式：发送请求给下一个服务器，对方给出一个可能知道答案的域名服务器
- 本地域名服务器一般支持递归解析方式，主机一般采用递归解析方式
- 本地域名服务器以及其他域名服务器进一步可采取迭代或递归方式
 - 由于递归查找会带来大量开销，实践中一般采用迭代方式



域名缓存Cache

- DNS客户、途中的解析DNS服务器缓存解析结果。收到新的请求在缓存查找时采用**最长后缀匹配**
- 缓存有一定的生存时间TTL
 - 最初的TTL由域名记录的拥有者控制
 - 在缓存中保存时相应地减少，TTL为0时从缓存中移走
- 成功的DNS响应被缓存
 - 下次查询同一域名可以快速响应
 - 其他查询可以用到本次查询得到的部分结果，比如NS记录
- DNS响应报告**不存在该域名时**也可以缓存

- 缓存提高了解析的速度，减少域名服务器的开销，提高了可靠性
- 缓存的RR记录可能已经过时，不一致
 - RR记录已经更改，但域名缓存中的RR记录仍然使用
 - 直到缓存中的RR记录过期才会一致
 - best-effort的域名解析



DNS 协议

- DNS服务器采用端口号53，支持UDP和TCP协议
- （主从域名服务器之间的）区域传输一般采用TCP
- DNS查询一般采用UDP，无需建立连接，可以支持更多的用户
 - UDP：无连接方式，提供不可靠的数据传输服务，保护数据的边界
 - DNS查询采用UDP，会由于丢失，收不到响应
 - DNS查询发送后设置超时，超时后等待一段时间重传
 - 可同时有多个查询进行，需要明确DNS响应回答的是哪个查询
 - DNS查询和DNS响应包括了ID字段
 - 检查ID字段以及响应中包含的问题是否就是查询中的问题
 - 采用UDP传输的DNS消息限制为512字节，如果超过512字节，多余的部分会被截掉
 - 可采用TCP再次查询
- DNS查询也可以采用TCP
 - TCP：面向连接方式，可靠的数据传输服务，将数据看成字节流，不保护数据的边界
 - 首先发送两个字节的整数，给出了接下来发送的DNS消息的长度

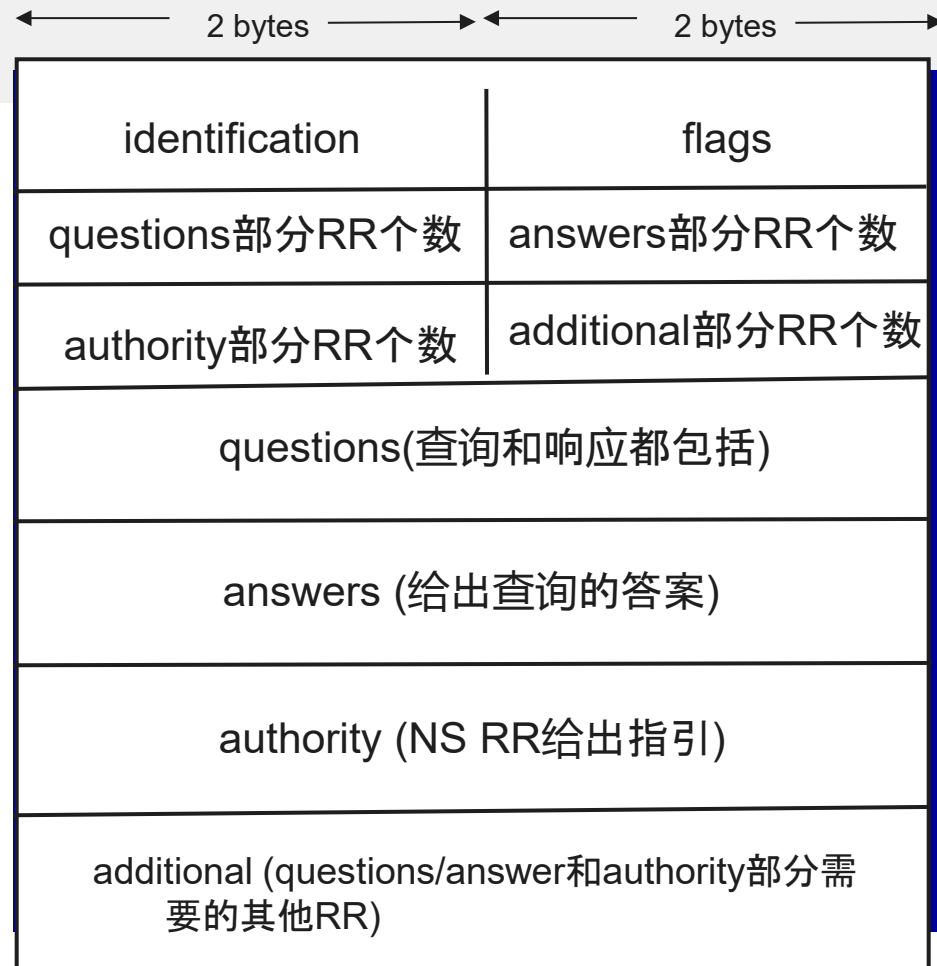
DNS消息

- DNS查询和响应格式一致
- identification: 查询和响应之间的对应

flags	QR	OPCODE	AA	TC	RD	RA	Z	RCODE
-------	----	--------	----	----	----	----	---	-------

QR	查询=0, 响应=1
OPCODE	4位, 描述操作方式: 标准查询, 反向查询(可选), UPDATE等
AA	Authoritative Answer, =1表示来自于权威域名服务器
TC	Truncation, 表示消息是否截取 (超过512字节)
RD	Recursion Desired, 发送查询时设置, =1 希望采用递归查询
RA	Recursion Available, =1 服务器支持递归查询
RCODE	4位返回码, 是否成功。如果失败, 给出了失败的原因
Z	保留, 全0, DNSSEC进行了扩展

- questions部分包括(NAME, TYPE, CLASS)
- 其他部分的RR记录包含(NAME, TYPE, CLASS, TTL, VALUE)



RCODE的常用取值

- NOERROR: 成功
- NXDomain: 域名不存在

DNS消息示例

```
> User Datagram Protocol, Src Port: 65113, Dst Port: 53
▼ Domain Name System (query)
  Transaction ID: 0xc133
  ▼ Flags: 0x0100 Standard query
    0... .. = Response: Message is a query
    .000 0... .. = Opcode: Standard query (0)
    .... ..0. .... = Truncated: Message is not truncated
    .... ..1 .... = Recursion desired: Do query recursively
    .... ..0.. .... = Z: reserved (0)
    .... ..0 .... = Non-authenticated data: Unacceptable
  Questions: 1
  Answer RRs: 0
  Authority RRs: 0
  Additional RRs: 0
  ▼ Queries
    ▼ dns.weixin.qq.com: type A, class IN
      Name: dns.weixin.qq.com
      [Name Length: 17]
      [Label Count: 4]
      Type: A (Host Address) (1)
      Class: IN (0x0001)
```

```
> User Datagram Protocol, Src Port: 53, Dst Port: 65113
▼ Domain Name System (response)
  Transaction ID: 0xc133
  > Flags: 0x8180 Standard query response, No error
  Questions: 1
  Answer RRs: 3
  Authority RRs: 0
  Additional RRs: 0
  ▼ Queries
    ▼ dns.weixin.qq.com: type A, class IN
      Name: dns.weixin.qq.com
      [Name Length: 17]
      [Label Count: 4]
      Type: A (Host Address) (1)
      Class: IN (0x0001)
  ▼ Answers
    ▼ dns.weixin.qq.com: type CNAME, class IN, cname v6.dns.weixin.qq.com
      Name: dns.weixin.qq.com
      Type: CNAME (Canonical NAME for an alias) (5)
      Class: IN (0x0001)
      Time to live: 600
      Data length: 5
      CNAME: v6.dns.weixin.qq.com
    ▼ v6.dns.weixin.qq.com: type A, class IN, addr 123.151.190.252
      Name: v6.dns.weixin.qq.com
      Type: A (Host Address) (1)
      Class: IN (0x0001)
      Time to live: 600
      Data length: 4
      Address: 123.151.190.252
    ▼ v6.dns.weixin.qq.com: type A, class IN, addr 123.150.208.86
      Name: v6.dns.weixin.qq.com
      Type: A (Host Address) (1)
      Class: IN (0x0001)
      Time to live: 600
      Data length: 4
      Address: 123.150.208.86
```

- DDoS攻击
 - 带宽泛洪攻击根服务器
 - 攻击造成的影响比较小
 - 根服务器所在网络的防火墙：负载过滤
 - 本地域名服务器缓存了TLD服务器的NS记录
 - 带宽泛洪攻击TLD服务器或者更低级的域名服务器：相比根服务器要容易一些
- 伪装攻击
 - 伪造DNS响应，污染解析域名服务器的缓存
 - RFC 4033/4034/4035：DNSSEC(DNS Security Extension)，提供DNS 资源记录的认证和完整性支持